700 Perguntas e Respostas

Preparação para entrevistas técnicas

React (por nível) · TypeScript · GitHub · JavaScript
Shopify · Next.js · Vaga Fullstack React + Node.js

100 perguntas por tema, divididas em Básico, Intermediário e Avançado.
Respostas objetivas, pensadas para revisão rápida: aprofunde cada tópico com a documentação oficial.

Sumário

1. React **3**

Nível Básico (34 perguntas) . . . 3

Nível Intermediário (33 perguntas) . . . 5

Nível Avançado (33 perguntas) . . . 7

2. TypeScript **10**

Nível Básico (34 perguntas) . . . 10

Nível Intermediário (35 perguntas) . . . 12

Nível Avançado (31 perguntas) . . . 14

3. GitHub **17**

Nível Básico (34 perguntas) . . . 17

Nível Intermediário (33 perguntas) . . . 18

Nível Avançado (33 perguntas) . . . 20

4. JavaScript **24**

Nível Básico (34 perguntas) . . . 24

Nível Intermediário (33 perguntas) . . . 26

Nível Avançado (33 perguntas) . . . 28

5. Shopify **31**

Nível Básico (34 perguntas) . . . 31

Nível Intermediário (34 perguntas) . . . 33

Nível Avançado (32 perguntas) . . . 35

6. Next.js **38**

Nível Básico (34 perguntas) . . . 38

Nível Intermediário (34 perguntas) . . . 40

Nível Avançado (32 perguntas) . . . 42

7. Vaga Fullstack React + Node.js **45**

Nível Básico (34 perguntas) . . . 45

Nível Intermediário (33 perguntas) . . . 47

Nível Avançado (33 perguntas) . . . 49

1. React

Nível Básico (34 perguntas)

1. O que é React?
Uma biblioteca JavaScript declarativa, baseada em componentes, criada pelo Facebook (Meta) para construir interfaces de usuário.

2. O que é JSX?
Uma extensão de sintaxe que permite escrever estruturas parecidas com HTML dentro do JavaScript. É transformada em chamadas `React.createElement` (ou `jsx()`) pelo compilador.

3. O que é um componente?
Uma função (ou classe) reutilizável que recebe props e retorna elementos React descrevendo parte da UI.

4. Qual a diferença entre componente funcional e de classe?
Funcionais são funções simples que usam Hooks para estado e efeitos; os de classe usam `this`, `state` e métodos de ciclo de vida. Hoje o padrão é usar funcionais.

5. O que são props?
Dados passados do componente pai para o filho, somente leitura (imutáveis dentro do filho).

6. O que é state?
Dados internos de um componente que mudam ao longo do tempo e, ao serem alterados, provocam nova renderização.

7. Qual a diferença entre props e state?
Props vêm de fora e são imutáveis para o componente; state pertence ao componente e é atualizado por ele (via `setState`/`useState`).

8. O que é o Virtual DOM?
Uma representação em memória da árvore de elementos. O React compara versões (diffing) e aplica no DOM real só o que mudou.

9. O que é o `useState`?
Hook que declara uma variável de estado em componente funcional. Retorna `[valor, setValor]`.

10. O que é o `useEffect`?
Hook para executar efeitos colaterais (requisições, assinaturas, manipulação do DOM) após a renderização, controlado por um array de dependências.

11. O que significa o array de dependências do `useEffect`?
Lista de valores que, ao mudarem, fazem o efeito rodar de novo. Array vazio roda só na montagem; sem array roda a cada renderização.

12. Como fazer uma renderização condicional?
Com `if`, operador ternário (`cond ? A : B`), `&&` (`cond && <A/>`) ou retornando `null`.

13. Como renderizar listas?
Usando `array.map()` retornando elementos, cada um com uma `key` única e estável.

14. Para que serve a `key`?
Ajuda o React a identificar quais itens mudaram, foram adicionados ou removidos, otimizando a reconciliação.

15. **Por que não usar o índice do array como `key`?**
    Se a lista for reordenada, filtrada ou tiver inserções, o índice muda e pode causar bugs de estado e renders desnecessários. Prefira IDs estáveis.

16. **O que são Fragments?**
    `<>...</>` permite agrupar vários elementos sem adicionar um nó extra ao DOM.

17. **Como tratar eventos no React?**
    Com props em camelCase, como `onClick={handler}`, passando uma função (não uma chamada).

18. **O que é um componente controlado?**
    Input cujo valor é controlado pelo estado React (`value` + `onChange`).

19. **O que é um componente não controlado?**
    Input cujo valor é gerenciado pelo próprio DOM e lido via `ref` quando necessário.

20. **O que são refs?**
    Referências mutáveis (`useRef`) que dão acesso a um nó do DOM ou guardam valores que persistem entre renders sem causar re-render.

21. **O que é "lifting state up"?**
    Mover o estado para o ancestral comum mais próximo para que componentes irmãos o compartilhem via props.

22. **O que é `children`?**
    Prop especial com o conteúdo colocado entre as tags de abertura e fechamento do componente.

23. **Como estilizar componentes no React?**
    CSS comum com `className`, CSS Modules, styled-components/Emotion, Tailwind ou estilos inline via `style={{}}`.

24. **Por que `className` em vez de `class`?**
    Porque `class` é palavra reservada em JavaScript; o JSX usa nomes de propriedades do DOM.

25. **Por que o estado não deve ser alterado diretamente?**
    Mutar o estado não dispara re-render e quebra otimizações. Sempre use o setter com um novo valor/objeto.

26. **O setState é síncrono?**
    Não. Atualizações são agrupadas (batching) e aplicadas na próxima renderização; para depender do valor anterior use a forma funcional `setX(prev => ...)`.

27. **O que é Create React App e qual a alternativa atual?**
    Era uma ferramenta de scaffolding, hoje descontinuada. Alternativas: Vite, Next.js, Remix/React Router.

28. **O que é Strict Mode?**
    Componente de desenvolvimento (`<StrictMode>`) que ativa verificações extras e executa certos ciclos em dobro para expor efeitos impuros.

29. **O que é prop drilling?**
    Passar props por vários níveis intermediários só para chegar a um componente profundo. Resolve-se com Context, composição ou gerenciadores de estado.

30. **O que é Context API?**
    Mecanismo para compartilhar valores (tema, usuário) com toda uma subárvore sem passar props manualmente.

31. Como passar dados do filho para o pai?
O pai passa uma função callback via props e o filho a chama com os dados.

32. O que é um componente puro?
Um que, dadas as mesmas props e state, sempre retorna o mesmo JSX e não causa efeitos colaterais durante a renderização.

33. Como o React sabe quando renderizar de novo?
Quando o state do componente muda, quando as props mudam ou quando o componente pai re-renderiza (ou um Context consumido muda).

34. O que é o `ReactDOM.createRoot`?
API do React 18+ que cria a raiz de renderização para montar a aplicação, habilitando recursos concorrentes.

Nível Intermediário (33 perguntas)

35. Para que serve o `useMemo`?
Memoriza o resultado de um cálculo custoso, recalculando-o apenas quando as dependências mudam.

36. Para que serve o `useCallback`?
Memoriza a referência de uma função entre renders, útil para evitar re-renders de filhos memoizados ou estabilizar dependências.

37. O que faz o `React.memo`?
Memoriza um componente e evita re-render se as props não mudaram (comparação rasa).

38. Qual a diferença entre `useMemo` e `useCallback`?
`useMemo` memoriza um valor calculado; `useCallback(fn, deps)` equivale a `useMemo(() => fn, deps)`, memorizando a função.

39. Quando usar `useReducer` em vez de `useState`?
Quando o estado é complexo, tem várias sub-partes ou transições que dependem do estado anterior, centralizando a lógica em um reducer.

40. O que é um custom Hook?
Função que começa com `use` e reutiliza lógica com estado entre componentes, compondo outros Hooks.

41. Como funciona o cleanup do `useEffect`?
A função retornada pelo efeito roda antes do próximo efeito e na desmontagem, usada para cancelar assinaturas, timers e requisições.

42. Qual a diferença entre `useEffect` e `useLayoutEffect`?
`useLayoutEffect` roda de forma síncrona após mutações do DOM e antes da pintura (para medir layout); `useEffect` roda depois da pintura.

43. O que é `useRef` além de acessar o DOM?
Guarda qualquer valor mutável que persiste entre renders (ex.: id de timer, valor anterior) sem disparar re-render.

44. O que é reconciliação?
O algoritmo que compara a árvore anterior com a nova e determina o conjunto mínimo de mudanças no DOM.

45. Como funciona o diffing do React?
Assume que elementos de tipos diferentes geram árvores diferentes e usa `key` para casar itens em listas, rodando em O(n).

46. O que são Error Boundaries?
Componentes de classe que capturam erros de renderização dos filhos e exibem uma UI de fallback (`getDerivedStateFromError`/`componentDidCatch`).

47. O que é `React.lazy` e `Suspense`?
`lazy` carrega componentes sob demanda (code splitting); `Suspense` exibe um fallback enquanto o código ou dados carregam.

48. O que é code splitting?
Dividir o bundle em partes carregadas sob demanda para reduzir o tempo de carregamento inicial.

49. Como funciona o React Router?
Biblioteca de roteamento que associa URLs a componentes, com `Routes`, `Route`, `Link`, `useNavigate`, `useParams` e rotas aninhadas.

50. O que é um Higher-Order Component (HOC)?
Função que recebe um componente e retorna outro com comportamento adicional (ex.: `withAuth`). Hoje, Hooks costumam substituí-lo.

51. O que é o padrão Render Props?
Técnica em que um componente recebe uma função como prop para decidir o que renderizar, compartilhando lógica.

52. O que é composição de componentes?
Construir UIs combinando componentes menores via `children` e props, preferida à herança no React.

53. O que são Portals?
`createPortal` renderiza filhos em um nó do DOM fora da hierarquia do pai, útil para modais e tooltips.

54. Como gerenciar formulários no React?
Com componentes controlados e `useState`, ou bibliotecas como React Hook Form e Formik, combinadas a validação (Zod/Yup).

55. Como buscar dados em um componente?
Com `useEffect` + `fetch`, ou melhor, bibliotecas como TanStack Query/SWR, que cuidam de cache, revalidação e estados de loading/erro.

56. O que é o TanStack Query (React Query)?
Biblioteca para gerenciar estado do servidor: cache, deduplicação, refetch, paginação e mutações.

57. Qual a diferença entre estado local, global e do servidor?
Local vive em um componente; global é compartilhado pela app (Context, Redux, Zustand); do servidor é dado remoto com cache e sincronização (React Query).

58. O que é Redux e quando usar?
Container de estado previsível com store, actions e reducers. Útil para estado global complexo; hoje se usa o Redux Toolkit.

59. O que é Redux Toolkit?
Conjunto oficial de utilitários que simplifica o Redux (`configureStore`, `createSlice`, `createAsyncThunk`, RTK Query).

60. Qual a alternativa leve ao Redux?
Zustand, Jotai, Recoil ou o próprio Context + `useReducer` para casos simples.

61. Por que Context pode causar re-renders excessivos?
Todos os consumidores re-renderizam quando o valor do Provider muda. Divida contextos e memorize o valor.

62. O que é `forwardRef`?
Permite que um componente repasse uma `ref` recebida para um elemento filho. (No React 19, `ref` pode ser prop direta.)

63. O que é `useImperativeHandle`?
Personaliza o valor exposto ao pai via `ref`, expondo só métodos específicos.

64. Qual a diferença entre `useId` e gerar IDs manualmente?
`useId` gera IDs únicos e estáveis entre servidor e cliente, evitando erros de hidratação.

65. O que é hidratação (hydration)?
Processo em que o React anexa eventos e estado ao HTML renderizado no servidor, tornando-o interativo.

66. O que é SSR, SSG e CSR?
SSR renderiza no servidor a cada requisição; SSG gera HTML no build; CSR renderiza no navegador via JavaScript.

67. Como testar componentes React?
Com Jest ou Vitest e React Testing Library, simulando interações do usuário e verificando o que aparece na tela.

Nível Avançado (33 perguntas)

68. O que é React Fiber?
A reimplementação do núcleo do React (16+) que quebra a renderização em unidades de trabalho interrompíveis, permitindo priorização e concorrência.

69. O que é renderização concorrente?
Capacidade do React de preparar várias versões da UI, pausar ou descartar renders de baixa prioridade para manter a interface responsiva.

70. O que faz `useTransition`?
Marca atualizações de estado como não urgentes (transições), mantendo a UI responsiva e expondo `isPending`.

71. O que faz `useDeferredValue`?
Retorna uma versão "atrasada" de um valor, permitindo que partes custosas atualizem depois de atualizações urgentes.

72. O que é automatic batching no React 18?
Agrupamento automático de múltiplas atualizações de estado em uma só renderização, inclusive em promises, timeouts e eventos nativos.

73. O que são React Server Components (RSC)?
Componentes executados apenas no servidor, sem enviar seu JS ao cliente, podendo acessar dados diretamente e reduzindo o bundle.

74. Qual a diferença entre Server e Client Components?
Server Components rodam no servidor e não têm estado nem efeitos; Client Components (`'use client'`) rodam no navegador e podem usar Hooks e eventos.

75. O que são Server Actions?
Funções assíncronas executadas no servidor (`'use server'`) e invocadas de formulários ou eventos, sem criar endpoints manuais.

76. O que faz o Hook `use` do React 19?
Lê o valor de uma Promise ou Context durante a renderização, integrando-se ao Suspense e podendo ser chamado condicionalmente.

77. O que é `useOptimistic`?
Hook do React 19 para exibir um estado otimista imediato enquanto uma ação assíncrona está em andamento.

78. O que é `useActionState`?
Hook que gerencia o estado resultante de uma ação de formulário, expondo estado, função de ação e status pendente.

79. O que é o React Compiler?
Compilador que memoriza componentes e valores automaticamente, reduzindo a necessidade de `useMemo`, `useCallback` e `memo` manuais.

80. Como o React decide a ordem dos Hooks?
Pela ordem de chamada em cada render, guardada em uma lista ligada na fiber. Por isso não podem ser chamados condicionalmente.

81. O que são stale closures?
Funções que capturam valores antigos de state/props de um render passado. Resolva com dependências corretas, forma funcional do setter ou refs.

82. Como o React trata atualizações de prioridade?
Usando "lanes": eventos discretos têm alta prioridade, transições e trabalho em segundo plano têm baixa.

83. O que é streaming SSR?
Enviar o HTML em partes conforme Suspense boundaries resolvem, melhorando TTFB e percepção de velocidade.

84. O que é Selective Hydration?
Hidratar partes da página sob demanda/prioridade (ex.: a região clicada primeiro), sem bloquear a página toda.

85. Como depurar problemas de performance em React?
Com React DevTools Profiler, Chrome Performance, `why-did-you-render`, análise de bundle e Web Vitals.

86. O que é virtualização de listas?
Renderizar apenas os itens visíveis (react-window, TanStack Virtual), essencial para listas com milhares de itens.

87. Como projetar uma arquitetura escalável de pastas em React?
Organização por feature/domínio, componentes compartilhados em `ui`, camadas para dados/hooks/serviços e limites claros de importação.

88. O que é o padrão Compound Components?
Componentes que trabalham juntos compartilhando estado implícito (ex.: `Tabs`, `Tabs.List`, `Tabs.Panel`), geralmente via Context.

89. O que é o padrão Controlled Props / State Reducer?
Permitem que o consumidor controle ou sobrescreva o estado interno de um componente reutilizável, dando flexibilidade a design systems.

90. Como implementar autenticação em uma SPA React?

Tokens via cookies httpOnly ou fluxo OAuth/OIDC, rotas protegidas, contexto de usuário e refresh de tokens, evitando guardar tokens sensíveis em localStorage.

91. Como lidar com acessibilidade (a11y) em React?

HTML semântico, atributos ARIA, gerenciamento de foco, labels em formulários, navegação por teclado e testes com axe/eslint-plugin-jsx-a11y.

92. Como funciona o `useSyncExternalStore`?

Hook para assinar fontes de dados externas de forma segura com renderização concorrente, evitando "tearing". Bibliotecas de estado o utilizam.

93. O que causa erros de hidratação e como evitá-los?

Diferenças entre HTML do servidor e do cliente (datas, `window`, aleatoriedade). Evite com `useEffect` para dados só do cliente ou `suppressHydrationWarning`.

94. Como evitar waterfalls de requisições?

Disparar requisições em paralelo (`Promise.all`), buscar dados em nível de rota/servidor, usar prefetch e Suspense.

95. O que é micro frontend e como aplicar com React?

Dividir a aplicação em frontends independentes por time, integrados via Module Federation, iframes ou web components.

96. Quando NÃO usar `useMemo`/`useCallback`?

Quando o cálculo é barato ou não há filhos memoizados; a memorização tem custo e complexidade e pode ser prematura.

97. Como o React lida com eventos (event delegation)?

Usa SyntheticEvents, anexando listeners na raiz da aplicação (não em cada nó) e normalizando o comportamento entre navegadores.

98. Como estruturar testes em uma aplicação React grande?

Pirâmide: testes unitários de hooks/utilitários, testes de integração de componentes com RTL, mocks de rede com MSW e poucos testes E2E (Playwright/Cypress).

99. O que é `Suspense` com data fetching?

Permite que bibliotecas compatíveis suspendam a renderização enquanto os dados carregam, exibindo o fallback da boundary mais próxima.

100. Como migrar uma aplicação de classes para Hooks?

Incrementalmente, convertendo componentes folha primeiro, extraindo lógica de ciclo de vida para Hooks e mantendo ambos coexistindo.

2. TypeScript

Nível Básico (34 perguntas)

1. O que é TypeScript?
Um superconjunto tipado do JavaScript, desenvolvido pela Microsoft, que é compilado para JavaScript puro e detecta erros em tempo de desenvolvimento.

2. Quais as vantagens do TypeScript?
Detecção precoce de erros, melhor autocomplete e refatoração, código autodocumentado e manutenção mais segura em projetos grandes.

3. Quais são os tipos primitivos?
`string`, `number`, `boolean`, `null`, `undefined`, `symbol` e `bigint`.

4. O que é o tipo `any`?
Desativa a checagem de tipos para aquele valor. Deve ser evitado, pois remove a segurança do TypeScript.

5. O que é o tipo `unknown`?
Um tipo seguro para valores desconhecidos: é preciso restringi-lo (type narrowing) antes de usá-lo.

6. Qual a diferença entre `any` e `unknown`?
`any` permite qualquer operação sem checagem; `unknown` exige verificação de tipo antes do uso.

7. O que é o tipo `never`?
Representa valores que nunca ocorrem, como funções que sempre lançam erro ou ramos impossíveis em checagens exaustivas.

8. O que é o tipo `void`?
Indica que uma função não retorna valor útil.

9. Como declarar um array tipado?
`string[]` ou `Array<string>`.

10. O que são tuplas?
Arrays com tamanho e tipos fixos por posição, como `[string, number]`.

11. O que é uma interface?
Declaração que descreve a forma de um objeto (propriedades e métodos), podendo ser estendida e implementada por classes.

12. O que é um type alias?
Nome para qualquer tipo, criado com `type`, incluindo uniões, interseções, tuplas e primitivos.

13. Qual a diferença entre `interface` e `type`?
Interfaces suportam declaration merging e `extends`; `type` aceita uniões, tuplas e tipos condicionais. Para objetos, ambos funcionam bem.

14. O que é inferência de tipos?
Capacidade do compilador de deduzir o tipo a partir do valor atribuído, sem anotação explícita.

15. O que são union types?
Tipos que aceitam um entre vários, como `string | number`.

16. O que são intersection types?
Combinam vários tipos em um só, com `A & B`, exigindo todas as propriedades.

17. **O que são literal types?**
    Tipos que representam valores exatos, como `"admin"` ou `42`, usados em uniões como `"GET" | "POST"`.

18. **O que são propriedades opcionais?**
    Declaradas com `?`, como `nome?: string`; o valor pode ser `undefined`.

19. **O que é `readonly`?**
    Modificador que impede reatribuição de uma propriedade após a inicialização.

20. **O que é um enum?**
    Conjunto de constantes nomeadas. Podem ser numéricos ou de string; muitos preferem uniões de literais por serem mais leves.

21. **Como tipar uma função?**
    Anotando parâmetros e retorno: `function soma(a: number, b: number): number`.

22. **Como definir parâmetros opcionais e padrão?**
    `(a: number, b?: number)` para opcional e `(a: number, b = 10)` para valor padrão.

23. **O que é type assertion?**
    Informar ao compilador o tipo de um valor com `as`, como `valor as string`. Não faz conversão em runtime.

24. **O que é o operador `!` (non-null assertion)?**
    Afirma que um valor não é `null`/`undefined` (`obj!.x`). Use com cautela.

25. **O que é optional chaining `?.`?**
    Acessa propriedades com segurança, retornando `undefined` se o valor intermediário for nulo.

26. **O que é nullish coalescing `??`?**
    Retorna o lado direito apenas se o esquerdo for `null` ou `undefined` (diferente de `||`, que considera falsy).

27. **O que é o `tsconfig.json`?**
    Arquivo de configuração do compilador: opções como `target`, `module`, `strict`, `paths` e arquivos incluídos.

28. **O que faz a opção `strict`?**
    Habilita um conjunto de checagens rigorosas (`strictNullChecks`, `noImplicitAny`, etc.), recomendada em novos projetos.

29. **O que é `noImplicitAny`?**
    Gera erro quando o TypeScript infere `any` implicitamente por falta de anotação.

30. **O que é `strictNullChecks`?**
    Torna `null` e `undefined` tipos distintos, evitando erros de acesso a valores nulos.

31. **Como tipar classes?**
    Declarando propriedades com tipos e modificadores de acesso (`public`, `private`, `protected`) e implementando interfaces com `implements`.

32. **O que é um arquivo `.d.ts`?**
    Arquivo de declaração de tipos, sem implementação, que descreve bibliotecas JavaScript.

33. **O que é o `@types`?**
    Pacotes no npm (como `@types/node`) que fornecem declarações de tipo para bibliotecas escritas em JavaScript.

34. Como o TypeScript é executado?
É transpilado para JavaScript por `tsc` (ou esbuild/SWC/Babel) e o JS resultante roda em Node ou no navegador. Ferramentas como `tsx` e `ts-node` executam direto.

Nível Intermediário (35 perguntas)

35. O que são generics?
Permitem criar componentes, funções e tipos reutilizáveis que funcionam com vários tipos, como `function id<T>(x: T): T`.

36. Como restringir um generic?
Com `extends`: `<T extends { id: number }>`, exigindo que `T` tenha certa forma.

37. O que é `keyof`?
Produz uma união com as chaves de um tipo: `keyof {a: 1; b: 2}` resulta em `"a" | "b"`.

38. O que é `typeof` em contexto de tipos?
Obtém o tipo de uma variável ou valor existente: `type T = typeof obj`.

39. O que são mapped types?
Tipos que transformam cada propriedade de outro tipo: `{ [K in keyof T]: ... }`.

40. O que são conditional types?
Tipos que escolhem um resultado conforme uma condição: `T extends string ? "sim" : "não"`.

41. O que é a palavra-chave `infer`?
Dentro de tipos condicionais, captura um tipo para reutilizá-lo, como em `T extends Promise<infer U> ? U : T`.

42. O que é `Partial<T>`?
Utilitário que torna todas as propriedades de `T` opcionais.

43. O que é `Required<T>`?
Torna todas as propriedades obrigatórias.

44. O que é `Pick<T, K>`?
Cria um tipo com apenas as propriedades `K` de `T`.

45. O que é `Omit<T, K>`?
Cria um tipo com todas as propriedades de `T`, exceto `K`.

46. O que é `Record<K, V>`?
Cria um tipo de objeto com chaves `K` e valores `V`.

47. O que são `Readonly<T>`?
Torna todas as propriedades de `T` somente leitura.

48. O que é `ReturnType<T>`?
Extrai o tipo de retorno de uma função.

49. O que é `Parameters<T>`?
Extrai os tipos dos parâmetros de uma função como tupla.

50. O que é `Awaited<T>`?
Obtém o tipo resolvido de uma Promise, desembrulhando recursivamente.

51. O que é `NonNullable<T>`?
Remove `null` e `undefined` de `T`.

52. O que são `Exclude` e `Extract`?
`Exclude<T, U>` remove de `T` os membros atribuíveis a `U`; `Extract<T, U>` mantém só esses membros.

53. O que é type narrowing?
Refinar um tipo mais amplo para um mais específico usando `typeof`, `instanceof`, `in`, igualdade ou guards.

54. O que são type guards?
Expressões ou funções que garantem o tipo dentro de um bloco, como `typeof x === "string"`.

55. O que são type predicates?
Funções com retorno `x is Tipo` que informam ao compilador que o argumento é daquele tipo.

56. O que são discriminated unions?
Uniões de objetos com uma propriedade literal comum (ex.: `kind`) que permite narrowing seguro com `switch`.

57. Como fazer checagem exaustiva?
No `default` do `switch`, atribuir o valor a `never`; se faltar um caso, o compilador acusa erro.

58. O que é `as const`?
Faz o literal ser inferido como imutável e com os tipos mais específicos possíveis (`readonly`, literais).

59. Qual a diferença entre `interface extends` e interseção `&`?
`extends` detecta conflitos de propriedades com erro e é mais performático; `&` combina tipos e pode produzir `never` em conflitos.

60. O que é declaration merging?
Interfaces com o mesmo nome no mesmo escopo são mescladas em uma só, útil para estender tipos de bibliotecas.

61. O que são index signatures?
Definem o tipo de chaves dinâmicas: `{ [chave: string]: number }`.

62. O que são template literal types?
Tipos de string construídos por interpolação, como `on${Capitalize<string>}` (com crases dentro do tipo).

63. O que são overloads de função?
Várias assinaturas para a mesma função, descrevendo diferentes combinações de parâmetros e retornos.

64. O que é o modificador `abstract`?
Marca classes ou métodos que devem ser implementados por subclasses; classes abstratas não podem ser instanciadas.

65. Qual a diferença entre `private` e `#private`?
`private` só existe em tempo de compilação; `#campo` é privacidade real do JavaScript em runtime.

66. Como tipar um componente React com props?
Definindo uma interface/type de props e aplicando ao parâmetro: `function Botao({ label }: BotaoProps)`.

67. Como tipar `useState` no React?
Com generics: `useState<Usuario | null>(null)`.

68. Como tipar eventos no React?
Com tipos como `React.ChangeEvent<HTMLInputElement>` e `React.MouseEvent<HTMLButtonElement>`.

69. Como tipar uma resposta de API com segurança?
Definir tipos e validar em runtime com bibliotecas como Zod, já que tipos desaparecem na compilação.

Nível Avançado (31 perguntas)

70. Como criar um tipo `DeepPartial`?
`type DeepPartial<T> = { [K in keyof T]?: T[K] extends object ? DeepPartial<T[K]> : T[K] }`.

71. O que é covariância e contravariância?
Covariância permite usar subtipos onde se espera um supertipo (retornos); contravariância inverte (parâmetros, sob `strictFunctionTypes`).

72. O que é `strictFunctionTypes`?
Faz a checagem contravariante dos parâmetros de funções, impedindo atribuições inseguras.

73. O que são variadic tuple types?
Tuplas com spread de tipos genéricos (`[...T, ...U]`), permitindo tipar concatenação de argumentos.

74. O que são key remapping em mapped types?
Uso de `as` para renomear chaves: `{ [K in keyof T as "get" + Capitalize<K & string>]: () => T[K] }`.

75. O que são distributive conditional types?
Tipos condicionais com parâmetro "nu" se distribuem sobre uniões: `T extends U ? X : Y` é aplicado a cada membro.

76. Como evitar a distribuição em conditional types?
Envolvendo em tuplas: `[T] extends [U] ? ... : ...`.

77. O que são branded types (tipos opacos)?
Técnica de adicionar uma marca fantasma (`string & { __brand: "UserId" }`) para diferenciar tipos estruturalmente idênticos.

78. Como tipar uma função que preserva literais?
Usando generics com `const` (`<const T>`) ou restrições como `T extends string`.

79. O que é `unique symbol`?
Tipo para símbolos únicos, útil para chaves e brands que nunca colidem.

80. O que é o `this` tipado?
É possível declarar o tipo de `this` como primeiro parâmetro falso: `function f(this: Contexto)`; também existe o tipo polimórfico `this`.

81. O que são decorators?
Funções anotadas com `@` que modificam classes e membros. O TS 5 suporta decorators do padrão ECMAScript; `experimentalDecorators` mantém o formato antigo.

82. O que é `module augmentation`?
Estender tipos de módulos existentes com `declare module "x" { ... }`, por exemplo, adicionando campos ao `Request` do Express.

83. O que é `declare global`?
Permite adicionar ou alterar tipos no escopo global a partir de um módulo.

84. O que é `isolatedModules`?
Exige que cada arquivo possa ser transpilado isoladamente, necessário para esbuild/SWC/Babel; restringe re-exports de tipos sem `type`.

85. Qual a diferença entre `tsc` e `esbuild`/`swc`?
`tsc` faz checagem de tipos e emite JS; esbuild/SWC apenas removem tipos e transpilam, muito mais rápido, sem checagem.

86. O que são project references?
Recurso de monorepos que divide o projeto em subprojetos com builds incrementais e dependências tipadas (`composite`).

87. Como funciona a resolução de módulos (`moduleResolution`)?
Define como imports são localizados: `node10`, `node16/nodenext` (respeita ESM/CJS e `exports`) e `bundler`.

88. Como tipar corretamente um `Object.keys`?
Retorna `string[]` por segurança estrutural; usa-se cast `as (keyof T)[]` quando se sabe a forma exata.

89. O que são recursive types?
Tipos que se referenciam, como `type Json = string | number | boolean | null | Json[] | { [k: string]: Json }`.

90. Como inferir tipos a partir de um schema Zod?
Com `z.infer<typeof schema>`, mantendo validação e tipo em uma única fonte.

91. O que é o tipo `Function` e por que evitá-lo?
Aceita qualquer função sem informar parâmetros ou retorno. Prefira assinaturas específicas como `(x: number) => void`.

92. Como funciona o `infer` com tuplas e funções?
Permite extrair partes, por exemplo `T extends (...args: infer A) => infer R ? [A, R] : never`.

93. Como tipar HOCs ou funções de composição de forma genérica?
Com generics que preservam as props originais e removem/injetam props via `Omit` e interseções.

94. O que é "type widening"?
Alargamento de tipos literais para o tipo base (`let x = "a"` vira `string`), enquanto `const x = "a"` mantém o literal.

95. Como lidar com bibliotecas sem tipos?
Instalar `@types/...`, criar um `.d.ts` com `declare module "lib"` ou usar `any` isolado como última opção.

96. Como reduzir o tempo de compilação em projetos grandes?
Usar `incremental`, project references, `skipLibCheck`, evitar tipos excessivamente complexos e usar checagem em CI separada do build.

97. O que são `abstract construct signatures`?
Assinaturas `abstract new () => T` que descrevem construtores de classes abstratas.

98. O que é o operador `using` (TS 5.2)?
Declaração de recursos descartáveis, chamando automaticamente `[Symbol.dispose]` ao sair do escopo.

99. Como tipar um event emitter de forma segura?

Com um mapa de eventos (`interface Eventos { login: User }`) e generics `on<K extends keyof Eventos>(evento: K, cb: (payload: Eventos[K]) => void)`.

100. Quando usar `unknown` em `catch`?

Com `useUnknownInCatchVariables` (parte do `strict`), o erro é `unknown`, obrigando a verificar com `instanceof Error` antes de usar.

3. GitHub

Nível Básico (34 perguntas)

1. **O que é Git?**
   Sistema de controle de versão distribuído que registra o histórico de alterações do código.

2. **O que é GitHub?**
   Plataforma de hospedagem de repositórios Git com recursos de colaboração: pull requests, issues, Actions, projetos e revisão de código.

3. **Qual a diferença entre Git e GitHub?**
   Git é a ferramenta de versionamento local; GitHub é um serviço online que hospeda repositórios Git e adiciona colaboração.

4. **O que é um repositório?**
   Pasta de projeto com todo o histórico de versões e configurações do Git.

5. **O que é um commit?**
   Um "snapshot" das alterações, com autor, data, mensagem e hash único.

6. **O que é uma branch?**
   Uma linha independente de desenvolvimento, um ponteiro móvel para um commit.

7. **Como clonar um repositório?**
   `git clone <url>`.

8. **Como criar uma nova branch e mudar para ela?**
   `git switch -c nome-da-branch` (ou `git checkout -b nome-da-branch`).

9. **O que faz `git add`?**
   Adiciona alterações à área de preparação (staging) para o próximo commit.

10. **O que faz `git commit -m`?**
    Cria um commit com as alterações em staging e a mensagem informada.

11. **O que faz `git push`?**
    Envia commits locais para o repositório remoto.

12. **O que faz `git pull`?**
    Busca alterações do remoto e as integra na branch atual (`fetch` + `merge`, ou `rebase` se configurado).

13. **Qual a diferença entre `git fetch` e `git pull`?**
    `fetch` só baixa as novidades sem alterar sua branch; `pull` baixa e integra.

14. **O que é um fork?**
    Cópia de um repositório para a sua conta, usada para contribuir com projetos de terceiros.

15. **O que é um Pull Request (PR)?**
    Solicitação para mesclar alterações de uma branch em outra, permitindo revisão, discussão e checks automatizados.

16. **O que é uma Issue?**
    Registro para rastrear bugs, tarefas, ideias e discussões em um repositório.

17. **O que é o `.gitignore`?**
    Arquivo que lista padrões de arquivos e pastas que o Git deve ignorar (ex.: `node_modules`, `.env`).

18. **O que é o `README.md`?**
    Documento de apresentação do projeto, exibido na página inicial do repositório.

19. **O que é o `origin`?**
    Nome padrão do repositório remoto de onde você clonou.

20. **Como ver o histórico de commits?**
    `git log` (ou `git log --oneline --graph`).

21. **Como ver o estado dos arquivos?**
    `git status`.

22. **Como ver as diferenças nas alterações?**
    `git diff` (não preparadas) e `git diff --staged` (preparadas).

23. **O que é `HEAD`?**
    Ponteiro para o commit atual em que você está (geralmente a ponta da branch ativa).

24. **O que é a branch `main`?**
    Branch principal padrão atual dos repositórios (antes comumente chamada `master`).

25. **Como desfazer alterações não commitadas em um arquivo?**
    `git restore <arquivo>` (antigo `git checkout -- arquivo`).

26. **Como remover um arquivo da staging?**
    `git restore --staged <arquivo>`.

27. **Como alterar a última mensagem de commit?**
    `git commit --amend` (evite se já deu push em branch compartilhada).

28. **O que é uma tag?**
    Marcação de um commit específico, comumente usada para versões como `v1.0.0`.

29. **O que é um Release no GitHub?**
    Publicação de uma versão baseada em tag, com notas e arquivos anexos.

30. **O que é o GitHub Pages?**
    Serviço gratuito para hospedar sites estáticos diretamente de um repositório.

31. **Como autenticar com o GitHub via terminal?**
    Com SSH keys, Personal Access Tokens ou GitHub CLI (`gh auth login`); senhas não são mais aceitas.

32. **O que é o GitHub CLI (`gh`)?**
    Ferramenta de linha de comando para criar PRs, issues e gerenciar repositórios sem sair do terminal.

33. **O que é um repositório público e um privado?**
    Público pode ser visto por qualquer pessoa; privado só por pessoas autorizadas.

34. **O que é uma licença open source?**
    Documento (MIT, Apache 2.0, GPL...) que define como terceiros podem usar, modificar e distribuir o código.

Nível Intermediário (33 perguntas)

35. **Qual a diferença entre `merge` e `rebase`?**
    `merge` une históricos criando um commit de mesclagem; `rebase` reaplica commits sobre outra base, gerando histórico linear, mas reescrevendo-o.

36. **Quando não usar `rebase`?**
    Em branches públicas/compartilhadas, pois reescreve o histórico e causa problemas para outros colaboradores.

37. **O que é "squash and merge"?**
    Opção do GitHub que combina todos os commits do PR em um único commit na branch destino.

38. **O que é `git stash`?**
    Guarda temporariamente alterações não commitadas para limpar o diretório de trabalho; `git stash pop` as recupera.

39. **O que é `git cherry-pick`?**
    Aplica na branch atual um commit específico de outra branch.

40. **Qual a diferença entre `git reset` e `git revert`?**
    `reset` move a branch para trás reescrevendo o histórico; `revert` cria um novo commit que desfaz outro, seguro para branches compartilhadas.

41. **Diferença entre `reset --soft`, `--mixed` e `--hard`?**
    `--soft` mantém alterações em staging; `--mixed` as mantém no diretório; `--hard` descarta tudo.

42. **O que é `git reflog`?**
    Registro local dos movimentos do HEAD, útil para recuperar commits "perdidos".

43. **O que é Git Flow?**
    Estratégia de branches com `main`, `develop`, `feature/*`, `release/*` e `hotfix/*`.

44. **O que é Trunk-Based Development?**
    Estratégia em que todos integram frequentemente em uma branch principal com branches curtas e feature flags.

45. **O que é GitHub Flow?**
    Fluxo simples: criar branch a partir da `main`, commitar, abrir PR, revisar, mesclar e fazer deploy.

46. **O que é Conventional Commits?**
    Padrão de mensagens (`feat:`, `fix:`, `chore:`...) que facilita changelogs e versionamento semântico automatizado.

47. **O que é versionamento semântico (SemVer)?**
    Formato `MAJOR.MINOR.PATCH`: major para quebras de compatibilidade, minor para novas funções e patch para correções.

48. **O que é o GitHub Actions?**
    Plataforma de CI/CD integrada que executa workflows definidos em YAML em resposta a eventos do repositório.

49. **Como é estruturado um workflow do GitHub Actions?**
    Arquivo `.github/workflows/*.yml` com `on` (gatilhos), `jobs`, `runs-on` e `steps` (comandos ou actions reutilizáveis).

50. **O que são Secrets no GitHub?**
    Variáveis criptografadas para guardar dados sensíveis (tokens, chaves) usadas em workflows sem expô-las no código.

51. **O que é CI/CD?**
    Integração Contínua (build e testes automáticos a cada mudança) e Entrega/Implantação Contínua (deploy automatizado).

52. O que são Branch Protection Rules?
Regras que exigem PRs, aprovações, checks passando e bloqueiam force push na branch.

53. O que é o CODEOWNERS?
Arquivo que define responsáveis por partes do código, solicitados automaticamente como revisores nos PRs.

54. O que são Pull Request Templates?
Modelos de descrição de PR (`.github/pull_request_template.md`) que padronizam as informações.

55. O que é um Draft Pull Request?
PR marcado como rascunho, sinalizando que ainda não está pronto para revisão ou merge.

56. Como manter um fork atualizado?
Adicionar o repositório original como `upstream`, fazer `git fetch upstream` e `merge`/`rebase`, ou usar "Sync fork" no GitHub.

57. O que são submodules?
Referências a outros repositórios dentro de um repositório, fixadas em um commit específico.

58. O que é `git bisect`?
Busca binária no histórico para encontrar o commit que introduziu um bug.

59. O que é `git blame`?
Mostra quem alterou cada linha de um arquivo pela última vez e em qual commit.

60. O que é Dependabot?
Recurso do GitHub que cria PRs automáticos para atualizar dependências e corrigir vulnerabilidades.

61. O que são GitHub Projects?
Quadros e tabelas para planejar e rastrear trabalho, vinculando issues e PRs.

62. O que é GitHub Discussions?
Fórum dentro do repositório para perguntas, ideias e conversas fora de issues.

63. Como assinar commits?
Com GPG ou SSH (`git commit -S`), exibindo o selo "Verified" no GitHub.

64. Como reverter um PR já mesclado?
Usar o botão "Revert" do GitHub, que cria um novo PR desfazendo as alterações.

65. O que é `git rebase -i`?
Rebase interativo para reordenar, editar, combinar (`squash`) ou remover commits.

66. O que é `--force-with-lease`?
Força o push apenas se o remoto não tiver novos commits que você desconhece, mais seguro que `--force`.

67. Como remover um arquivo sensível já commitado?
Reescrever o histórico com `git filter-repo` ou BFG e rotacionar imediatamente a credencial exposta.

Nível Avançado (33 perguntas)

68. O que é um workflow reutilizável (`workflow_call`)?
Workflow que pode ser chamado por outros, evitando duplicação de pipelines entre repositórios.

69. O que são composite actions?
Actions que agrupam vários passos em uma só, definidas em YAML sem precisar de código JavaScript ou Docker.

70. Como usar matrix strategy no Actions?
Com `strategy.matrix` para executar o mesmo job em várias combinações (versões do Node, sistemas operacionais).

71. Como acelerar workflows?
Cache de dependências (`actions/cache`), jobs paralelos, `concurrency` para cancelar execuções antigas e runners adequados.

72. O que é `concurrency` no Actions?
Garante que apenas uma execução de um grupo rode por vez e permite cancelar as anteriores.

73. O que são Environments no GitHub?
Ambientes (staging, production) com secrets próprios, revisores obrigatórios e regras de proteção para deploy.

74. Como autenticar o Actions na nuvem sem segredos fixos?
Via OIDC, em que o GitHub emite tokens de curta duração aceitos por AWS, Azure e GCP.

75. O que são self-hosted runners?
Máquinas próprias que executam jobs de Actions, úteis para hardware específico ou rede privada, exigindo cuidados de segurança.

76. Como proteger-se de ataques de supply chain em Actions?
Fixar actions por SHA de commit, usar permissões mínimas em `GITHUB_TOKEN`, revisar dependências e evitar executar código de PRs externos com secrets.

77. O que é `GITHUB_TOKEN`?
Token temporário gerado automaticamente a cada execução, com permissões configuráveis no escopo do repositório.

78. O que são Rulesets?
Evolução das regras de proteção: permitem regras em vários repositórios, com maior flexibilidade e modos de teste (evaluate).

79. O que é Merge Queue?
Fila que testa PRs com a versão mais recente da `main` antes de mesclar, evitando quebras por merges concorrentes.

80. O que é GitHub Advanced Security?
Conjunto com code scanning (CodeQL), secret scanning e revisão de dependências para detectar vulnerabilidades.

81. O que é CodeQL?
Mecanismo de análise estática que consulta o código como dados para encontrar vulnerabilidades.

82. O que é secret scanning com push protection?
Detecta credenciais em commits e bloqueia o push antes que o segredo chegue ao repositório.

83. Como gerenciar monorepos com GitHub?
Usar CODEOWNERS por pasta, workflows com filtros `paths`, ferramentas como Nx/Turborepo e builds incrementais com cache.

84. Como disparar workflows por eventos externos?
Com `repository_dispatch`, `workflow_dispatch` (manual) ou webhooks via API.

85. O que são GitHub Apps e OAuth Apps?
Integrações com a API: GitHub Apps têm permissões granulares e agem por instalação; OAuth Apps agem em nome do usuário com escopos amplos.

86. O que são webhooks do GitHub?
Notificações HTTP enviadas a uma URL quando eventos ocorrem (push, PR, issue), permitindo integrações.

87. Qual a diferença entre a API REST e a GraphQL do GitHub?
A REST usa múltiplos endpoints fixos; a GraphQL permite consultar exatamente os campos necessários em uma única requisição.

88. Como o Git armazena dados internamente?
Como objetos endereçados por hash (blobs, trees, commits, tags) em `.git/objects`, formando um grafo acíclico.

89. O que é uma "detached HEAD"?
Estado em que o HEAD aponta para um commit específico, não para uma branch; novos commits podem ser perdidos se não criar uma branch.

90. O que é `git worktree`?
Permite ter várias pastas de trabalho do mesmo repositório com branches diferentes simultaneamente.

91. O que é sparse checkout?
Baixar apenas parte dos arquivos do repositório, útil em monorepos grandes.

92. O que é shallow clone?
Clone com histórico limitado (`--depth 1`), mais rápido e leve, comum em CI.

93. O que é Git LFS?
Extensão que armazena arquivos grandes (binários, mídia) fora do repositório principal, mantendo apenas ponteiros.

94. Como lidar com histórico muito grande ou lento?
Usar `git gc`, shallow clones, partial clone (`--filter=blob:none`), LFS e remover arquivos grandes do histórico.

95. O que é `git rerere`?
"Reuse recorded resolution": memoriza como você resolveu conflitos para reaplicar automaticamente em situações iguais.

96. O que é uma estratégia de release com branches de manutenção?
Manter branches como `release/1.x` para receber correções (backports via cherry-pick) enquanto o desenvolvimento continua na `main`.

97. Como automatizar releases e changelogs?
Com Conventional Commits e ferramentas como semantic-release ou release-please, acionadas por Actions.

98. Como configurar deploy contínuo seguro com GitHub Actions?
Ambientes protegidos com aprovação, OIDC, permissões mínimas, deploys reversíveis (rollback) e checks obrigatórios antes de produção.

99. Como medir a saúde de um fluxo de engenharia?
Métricas DORA: frequência de deploy, lead time, taxa de falhas e tempo de restauração.

100. Como estruturar a governança de uma organização no GitHub?

Times e permissões por papel, SSO/SAML, rulesets, CODEOWNERS, políticas de segurança e auditoria via audit log.

4. JavaScript

Nível Básico (34 perguntas)

1. O que é JavaScript?
Linguagem de programação interpretada, dinâmica e multiparadigma, usada no navegador, no servidor (Node.js) e em muitos outros ambientes.

2. Quais são os tipos de dados primitivos?
`string`, `number`, `boolean`, `null`, `undefined`, `symbol` e `bigint`.

3. Qual a diferença entre `var`, `let` e `const`?
`var` tem escopo de função e sofre hoisting com valor `undefined`; `let` e `const` têm escopo de bloco; `const` não permite reatribuição.

4. O que é hoisting?
Comportamento em que declarações são "elevadas" ao topo do escopo antes da execução. Funções são totalmente elevadas; `let`/`const` ficam na temporal dead zone.

5. O que é a Temporal Dead Zone?
Período entre o início do escopo e a declaração de `let`/`const`, em que acessar a variável gera `ReferenceError`.

6. Qual a diferença entre `==` e `===`?
`==` compara com coerção de tipos; `===` compara valor e tipo sem coerção. Prefira `===`.

7. O que são valores falsy?
`false`, `0`, `-0`, `0n`, `""`, `null`, `undefined` e `NaN`. Todo o resto é truthy.

8. O que é `typeof null`?
Retorna `"object"`, um bug histórico da linguagem mantido por compatibilidade.

9. O que é `NaN`?
"Not a Number", valor numérico resultante de operações inválidas. Só se verifica com `Number.isNaN()`.

10. O que é uma função?
Bloco reutilizável de código que pode receber parâmetros e retornar valor; em JS funções são valores de primeira classe.

11. Qual a diferença entre function declaration e function expression?
A declaração sofre hoisting completo e pode ser chamada antes de definida; a expressão só existe após a atribuição.

12. O que são arrow functions?
Sintaxe curta de função (`(a, b) => a + b`) que não tem `this`, `arguments` próprios e herda o `this` do escopo.

13. O que é um objeto?
Coleção de pares chave-valor, base da maioria dos dados em JavaScript.

14. Como criar e acessar propriedades de objetos?
Com literais `{ nome: "Ana" }`, acessando por `obj.nome` ou `obj["nome"]`.

15. O que é um array?
Objeto ordenado e indexado para armazenar listas de valores.

16. **Para que servem `map`, `filter` e `reduce`?**
    `map` transforma cada item; `filter` seleciona itens por condição; `reduce` acumula os itens em um único valor.

17. **Qual a diferença entre `forEach` e `map`?**
    `forEach` apenas executa algo para cada item e retorna `undefined`; `map` retorna um novo array transformado.

18. **Como adicionar e remover elementos de um array?**
    `push`/`pop` no final; `unshift`/`shift` no início; `splice` em qualquer posição.

19. **O que é template literal?**
    Strings com crases que permitem interpolação `${expressao}` e múltiplas linhas.

20. **O que é destructuring?**
    Sintaxe para extrair valores de arrays/objetos em variáveis: `const { nome } = pessoa`.

21. **O que são os operadores spread e rest?**
    `...` expande iteráveis/objetos (spread) ou agrupa argumentos restantes em um array (rest).

22. **O que é o DOM?**
    Representação em árvore do documento HTML que o JavaScript pode ler e manipular.

23. **Como selecionar elementos do DOM?**
    `document.querySelector`, `querySelectorAll`, `getElementById` e similares.

24. **O que são eventos e como tratá-los?**
    Ações como cliques ou teclas. Usa-se `elemento.addEventListener("click", handler)`.

25. **O que é `null` versus `undefined`?**
    `undefined` indica ausência de valor atribuído; `null` é uma ausência intencional.

26. **O que é um escopo?**
    Região do código em que uma variável é acessível: global, de função ou de bloco.

27. **O que é um closure?**
    Uma função que "lembra" das variáveis do escopo onde foi criada, mesmo depois de esse escopo ter terminado.

28. **O que é `this`?**
    Referência ao contexto de execução; seu valor depende de como a função é chamada (objeto, global, `new`, `call/apply/bind`).

29. **O que é JSON e como usá-lo?**
    Formato textual de dados. `JSON.stringify` converte objeto em string e `JSON.parse` faz o inverso.

30. **Como funciona o `try...catch`?**
    Executa o código no `try`; se ocorrer erro, o fluxo vai para o `catch`; `finally` roda sempre.

31. **O que é `setTimeout` e `setInterval`?**
    `setTimeout` executa uma função uma vez após um atraso; `setInterval` a executa repetidamente.

32. **O que é uma Promise?**
    Objeto que representa o resultado futuro de uma operação assíncrona: pendente, resolvida ou rejeitada.

33. **O que é `async/await`?**
    Sintaxe que permite escrever código assíncrono de forma sequencial; `await` pausa a função até a Promise resolver.

34. O que é callback?
Função passada como argumento para ser chamada depois, comum em operações assíncronas e eventos.

Nível Intermediário (33 perguntas)

35. O que é o Event Loop?
Mecanismo que coordena a call stack, as filas de tarefas e microtarefas, permitindo operações assíncronas em uma única thread.

36. Qual a diferença entre microtasks e macrotasks?
Microtasks (Promises, `queueMicrotask`) rodam após a tarefa atual, antes da próxima macrotask (`setTimeout`, eventos).

37. O que é a call stack?
Pilha que registra as funções em execução; estouro dela gera `Maximum call stack size exceeded`.

38. O que são prototypes?
Mecanismo de herança do JavaScript: objetos herdam propriedades de seu protótipo, formando a cadeia de protótipos.

39. Como funcionam as classes em JS?
São açúcar sintático sobre protótipos: `class`, `constructor`, `extends`, `super`, métodos estáticos e campos privados `#`.

40. Como funcionam `call`, `apply` e `bind`?
Definem o `this`: `call` chama com argumentos separados, `apply` com array, `bind` retorna nova função com `this` fixado.

41. O que é currying?
Transformar uma função de vários argumentos em uma sequência de funções de um argumento: `f(a)(b)`.

42. O que são funções de ordem superior?
Funções que recebem outras funções como argumento ou as retornam (`map`, `filter`, decoradores).

43. O que é uma função pura?
Dado o mesmo input, sempre retorna o mesmo output e não causa efeitos colaterais.

44. O que é imutabilidade e como obtê-la?
Não alterar dados existentes, criando cópias (spread, `map`, `structuredClone`) ou usando `Object.freeze`.

45. Diferença entre cópia rasa e profunda?
Cópia rasa (spread, `Object.assign`) copia só o primeiro nível; profunda (`structuredClone`) copia recursivamente.

46. Qual a diferença entre `Promise.all`, `allSettled`, `race` e `any`?
`all` falha se uma falhar; `allSettled` espera todas e informa cada resultado; `race` retorna a primeira que concluir; `any` a primeira que resolver.

47. Como tratar erros em async/await?
Com `try/catch` ao redor do `await` ou `.catch()` na Promise.

48. O que é `fetch`?
API nativa baseada em Promises para requisições HTTP; não rejeita em respostas 4xx/5xx, é preciso checar `response.ok`.

49. O que é CORS?
Mecanismo de segurança do navegador que controla requisições entre origens diferentes, configurado por cabeçalhos no servidor.

50. O que é debounce?
Atrasar a execução de uma função até que um tempo sem novas chamadas passe (ex.: busca ao digitar).

51. O que é throttle?
Limitar a execução de uma função a no máximo uma vez por intervalo de tempo (ex.: scroll).

52. O que é event bubbling e capturing?
Propagação de eventos: captura desce da raiz ao alvo; bubbling sobe do alvo à raiz. `stopPropagation` interrompe.

53. O que é event delegation?
Registrar um listener no ancestral para tratar eventos de vários filhos, aproveitando o bubbling.

54. O que são generators?
Funções (`function*`) que pausam com `yield` e retornam iteradores, úteis para sequências preguiçosas.

55. O que são iteradores e iteráveis?
Iterável implementa `[Symbol.iterator]`; iterador tem `next()` que retorna `{ value, done }`. Usados por `for...of` e spread.

56. Qual a diferença entre `for...in` e `for...of`?
`for...in` itera sobre chaves enumeráveis de objetos; `for...of` itera sobre valores de iteráveis.

57. O que são Map e Set?
`Map` é coleção chave-valor com qualquer tipo de chave; `Set` guarda valores únicos.

58. O que são WeakMap e WeakSet?
Coleções com referências fracas a objetos, permitindo coleta de lixo; não são iteráveis.

59. O que é `Symbol`?
Tipo primitivo que cria identificadores únicos, usado em chaves de propriedades e protocolos como `Symbol.iterator`.

60. O que é optional chaining e nullish coalescing?
`?.` acessa propriedades com segurança; `??` define um padrão apenas para `null`/`undefined`.

61. Para que serve `Object.freeze` e `Object.seal`?
`freeze` torna o objeto imutável (raso); `seal` impede adicionar ou remover propriedades, mas permite alterá-las.

62. O que é `Proxy`?
Objeto que intercepta e redefine operações (leitura, escrita, etc.) de outro objeto.

63. Como funciona o garbage collector?
Libera memória de objetos inalcançáveis, usando algoritmos como mark-and-sweep.

64. O que causa memory leaks em JS?
Variáveis globais acidentais, listeners e timers não removidos, closures retendo dados grandes e referências a nós do DOM removidos.

65. O que é `Intl`?
API de internacionalização para formatar datas, números, moedas e comparar strings conforme o idioma.

66. Como trabalhar com datas em JS?
Com `Date`, `Intl.DateTimeFormat`, ou bibliotecas como date-fns/Day.js; a futura API `Temporal` resolverá muitas limitações.

67. O que são módulos CommonJS e ES Modules?
CommonJS usa `require`/`module.exports` (síncrono, Node clássico); ES Modules usam `import`/`export` (estáticos, padrão da linguagem).

Nível Avançado (33 perguntas)

68. Explique a ordem de execução de `console.log`, `setTimeout(0)` e `Promise.then`.
Código síncrono primeiro, depois microtasks (`then`), e por último macrotasks (`setTimeout`), mesmo com atraso 0.

69. O que é `Promise.withResolvers`?
Método que retorna `{ promise, resolve, reject }`, simplificando a criação de promises controladas externamente.

70. Como implementar `Promise.all` do zero?
Retornar nova Promise, iterar as promises guardando resultados por índice, resolver quando o contador chegar ao total e rejeitar na primeira falha.

71. Como funciona a herança prototipal em detalhes?
Ao acessar uma propriedade, o motor busca no objeto e sobe pela cadeia `__proto__` até `null`; `Object.create` define o protótipo explicitamente.

72. O que é a diferença entre `Object.create(null)` e `{}`?
O primeiro cria objeto sem protótipo (sem `toString`, etc.), útil como dicionário puro.

73. O que são property descriptors?
Metadados de propriedades (`value`, `writable`, `enumerable`, `configurable`, `get`, `set`), manipulados por `Object.defineProperty`.

74. O que é o módulo `Reflect`?
API com métodos que espelham operações de objetos, usada em conjunto com `Proxy`.

75. O que é tail call optimization em JS?
Otimização que reaproveita o frame de chamadas em posição final; faz parte da especificação, mas só é suportada por alguns motores.

76. Como funcionam `SharedArrayBuffer` e `Atomics`?
Permitem memória compartilhada entre workers, com operações atômicas para evitar race conditions.

77. O que são async iterators e `for await...of`?
Iteradores que retornam Promises, usados para consumir fluxos assíncronos de dados.

78. Como funcionam `AbortController` e `AbortSignal`?
Permitem cancelar operações assíncronas, como `fetch`, sinalizando via `controller.abort()`.

79. O que é Event Loop no Node.js vs navegador?
Node usa libuv com fases (timers, I/O, `setImmediate`, close), além de `process.nextTick`; o navegador tem filas de tarefas e renderização.

80. O que é Tree Shaking?
Remoção de código não utilizado durante o bundling, dependendo de ES Modules estáticos e ausência de efeitos colaterais.

81. Como funciona o motor V8?
Compila JS em bytecode (Ignition) e otimiza código "quente" com JIT (TurboFan), usando hidden classes e inline caching.

82. O que são hidden classes?
Estruturas internas do V8 que representam o "formato" de objetos; manter a mesma ordem de propriedades melhora a performance.

83. Como identificar e resolver gargalos de performance no navegador?
Com a aba Performance e Lighthouse do DevTools, reduzindo trabalho na main thread, layout thrashing e tamanho de bundles.

84. O que é layout thrashing?
Leituras e escritas alternadas de layout no DOM que forçam recálculos repetidos; evite agrupando leituras e escritas.

85. O que são `requestAnimationFrame` e `requestIdleCallback`?
`rAF` executa antes da próxima pintura (animações); `rIC` executa em momentos ociosos para trabalho de baixa prioridade.

86. O que é um Service Worker?
Script que atua como proxy entre app e rede, habilitando cache offline, push notifications e PWAs.

87. O que é XSS e como prevenir?
Injeção de scripts em páginas. Previna sanitizando/escapando entradas, evitando `innerHTML` com dados não confiáveis e usando CSP.

88. O que é CSRF?
Ataque que força o navegador autenticado a executar ações indesejadas; previne-se com tokens CSRF e cookies `SameSite`.

89. O que é prototype pollution?
Vulnerabilidade em que um invasor altera `Object.prototype` via chaves como `__proto__`, afetando toda a aplicação.

90. Como funciona `structuredClone`?
Cria cópia profunda de valores clonáveis (Map, Set, Date, ciclos), mas não copia funções nem protótipos de classes.

91. O que é o padrão Observer?
Objetos (observers) se inscrevem em um sujeito e são notificados de mudanças; base de eventos e streams.

92. O que é o padrão Module e IIFE?
IIFE é função executada imediatamente, criando escopo privado; o padrão Module usa closures para encapsular estado, hoje substituído por ES Modules.

93. O que é memoization?
Cache do resultado de funções puras com base nos argumentos, evitando recálculo.

94. Como lidar com precisão em números decimais?
Números são float de 64 bits (`0.1 + 0.2 !== 0.3`); use inteiros em centavos, `toFixed` na exibição ou bibliotecas decimais; `BigInt` para inteiros grandes.

95. O que são Tagged Templates?
Funções que processam template literals (`tag` seguida de um template literal), usadas em styled-components e SQL seguros.

96. O que são decorators e o estado atual no JavaScript?

Proposta padronizada para anotar classes e membros com `@`; já é suportada por transpiladores e vem sendo implementada nos motores.

97. Qual a diferença entre `import()` dinâmico e import estático?

O dinâmico carrega módulos sob demanda e retorna Promise, permitindo code splitting; o estático é resolvido antes da execução.

98. O que é top-level await?

Permite usar `await` no topo de módulos ES, sem envolver em função async.

99. Como funciona a coerção de tipos em `[] + {}` e `[] == ![]`?

Operadores convertem para primitivos: `[] + {}` resulta em `"[object Object]"` e `[] == ![]` é `true`, pois ambos viram `0`. Evite coerção implícita.

100. Como garantir qualidade de código JavaScript em equipes?

ESLint, Prettier, testes (Vitest/Jest), revisão de código, TypeScript/JSDoc e CI automatizado.

5. Shopify

Nível Básico (34 perguntas)

1. O que é a Shopify?
Plataforma SaaS de e-commerce que permite criar e gerenciar lojas online, com hospedagem, checkout, pagamentos e logística integrados.

2. Quais são as principais vantagens da Shopify?
Rapidez para lançar a loja, hospedagem e segurança gerenciadas, ecossistema de apps e temas, checkout otimizado e escalabilidade.

3. O que é um tema (theme) na Shopify?
Conjunto de arquivos (Liquid, CSS, JS, JSON) que define a aparência e a estrutura da loja virtual.

4. O que é Liquid?
Linguagem de template open source criada pela Shopify, usada para renderizar dados dinâmicos (produtos, coleções, carrinho) nos temas.

5. Quais são os três tipos de delimitadores do Liquid?
`{{ }}` para saída de dados, `{% %}` para lógica (if, for, assign) e filtros com `|` aplicados a saídas.

6. O que são filtros no Liquid?
Funções que modificam a saída, como `{{ produto.title | upcase }}` ou `{{ preco | money }}`.

7. O que é o Online Store 2.0?
Arquitetura moderna de temas, com seções em todas as páginas, templates JSON e blocos de app, permitindo personalização sem código.

8. O que são seções (sections)?
Blocos reutilizáveis de conteúdo e layout, com configurações no editor de temas (arquivos em `/sections`).

9. O que são blocos (blocks)?
Elementos configuráveis dentro de uma seção, que o lojista pode adicionar, reordenar e remover no editor.

10. O que é o Theme Editor (Personalizador)?
Editor visual para configurar seções, blocos e estilos do tema sem mexer no código.

11. O que são templates JSON?
Arquivos que definem quais seções aparecem em cada tipo de página, em formato JSON, permitindo composição no editor.

12. Qual a estrutura de pastas de um tema?
`assets`, `config`, `layout`, `locales`, `sections`, `snippets`, `templates` e (no OS 2.0) `blocks`.

13. O que é um snippet?
Trecho de Liquid reutilizável, incluído com `{% render 'nome' %}`.

14. O que é o `theme.liquid`?
Arquivo de layout principal que envolve todas as páginas, contendo `<head>`, header, footer e `{{ content_for_layout }}`.

15. O que são produtos, variantes e coleções?
Produto é o item vendido; variantes são suas opções (tamanho, cor); coleções agrupam produtos manual ou automaticamente.

16. **Quantas opções e variantes um produto pode ter?**
    Tradicionalmente até 3 opções e 100 variantes; limites maiores existem em planos/recursos mais novos, então verifique a documentação atual.

17. **O que são metafields?**
    Campos personalizados que armazenam dados extras em produtos, coleções, clientes, pedidos e outros recursos.

18. **O que são metaobjects?**
    Tipos de dados estruturados e personalizados (ex.: "Designers", "FAQ") criados no admin e reutilizáveis na loja.

19. **O que é o Shopify Admin?**
    Painel de administração onde o lojista gerencia produtos, pedidos, clientes, marketing e configurações.

20. **O que é o Shopify Payments?**
    Gateway de pagamentos nativo da Shopify, disponível em alguns países. Em outros usam-se gateways de terceiros.

21. **O que é o Shopify POS?**
    Ponto de venda para vendas presenciais, sincronizado com estoque e pedidos online.

22. **Qual a diferença entre Shopify e WooCommerce?**
    Shopify é SaaS hospedado e gerenciado; WooCommerce é plugin do WordPress, auto-hospedado, com mais controle e mais responsabilidade técnica.

23. **O que é a Shopify App Store?**
    Marketplace de aplicativos que estendem funcionalidades como avaliações, e-mail marketing, SEO e frete.

24. **O que é o Shopify CLI?**
    Ferramenta de linha de comando para criar apps, desenvolver temas localmente (`shopify theme dev`) e fazer deploy.

25. **Como desenvolver um tema localmente?**
    Com Shopify CLI (`shopify theme dev`) para pré-visualizar, e integração com Git/GitHub para versionar.

26. **O que são os planos da Shopify?**
    Planos como Starter, Basic, Grow/Shopify e Advanced, além de Plus para grandes empresas, com diferenças em taxas, funcionalidades e relatórios.

27. **O que é o Shopify Plus?**
    Plano enterprise com maior personalização do checkout, automações (Flow), múltiplas lojas e suporte dedicado.

28. **O que é o objeto `product` no Liquid?**
    Objeto global na página de produto com propriedades como `title`, `price`, `variants`, `images`, `description`.

29. **Como exibir um preço formatado?**
    Com o filtro `money`: `{{ product.price | money }}`.

30. **Como fazer um loop de produtos de uma coleção?**
    `{% for product in collection.products %} ... {% endfor %}`.

31. **O que é o objeto `cart`?**
    Objeto com os itens do carrinho, total, quantidade e atributos.

32. O que é o Dawn?
Tema de referência gratuito e open source da Shopify, rápido e baseado em OS 2.0.

33. Como configurar um domínio próprio?
No Admin, em Configurações > Domínios, conectando o domínio por DNS (registros A/CNAME) ou comprando pela Shopify.

34. Como criar descontos?
Em Descontos no Admin: códigos ou automáticos, por porcentagem, valor fixo, frete grátis ou "compre X leve Y".

Nível Intermediário (34 perguntas)

35. O que é a Storefront API?
API GraphQL pública para criar experiências de compra personalizadas (headless, apps móveis) com produtos, carrinho e checkout.

36. O que é a Admin API?
API REST/GraphQL para gerenciar dados da loja (produtos, pedidos, clientes) a partir de apps e integrações, com autenticação por token.

37. Por que preferir GraphQL à REST na Shopify?
A GraphQL é o foco de novos recursos, permite buscar apenas os campos necessários e a Shopify marcou a REST Admin API como legada.

38. O que é rate limiting na API da Shopify?
Limites de requisições; a GraphQL usa custo calculado por query com "bucket" que se recupera ao longo do tempo, e a REST usa um modelo leaky bucket.

39. O que são webhooks?
Notificações HTTP enviadas pela Shopify quando eventos ocorrem (pedido criado, produto atualizado), exigindo verificação do HMAC.

40. Como validar um webhook da Shopify?
Calculando o HMAC SHA-256 do corpo bruto com o segredo do app e comparando com o cabeçalho `X-Shopify-Hmac-Sha256`.

41. Qual a diferença entre app público, personalizado e privado (custom)?
Público é listado na App Store; personalizado é desenvolvido para uma loja específica; apps criados no Dev Dashboard/Admin são instalados diretamente por lojistas.

42. Como funciona a autenticação OAuth para apps?
O lojista autoriza escopos, a Shopify devolve um código que o app troca por um access token. Apps embarcados usam também token exchange/session tokens.

43. O que são scopes (escopos) de acesso?
Permissões solicitadas pelo app (ex.: `read_products`, `write_orders`), concedidas pelo lojista na instalação.

44. O que são App Extensions?
Extensões que adicionam funcionalidades à loja ou ao admin sem alterar o tema: theme app extensions, checkout UI, functions, POS UI.

45. O que são Theme App Extensions?
Permitem que apps adicionem blocos e embeds ao tema via editor, sem editar código Liquid do lojista.

46. O que são Shopify Functions?
Código customizado (Rust, JavaScript/WASM) executado na infraestrutura da Shopify para personalizar descontos, frete, pagamentos e validações.

47. O que são Checkout UI Extensions?
Componentes React/JS que adicionam elementos personalizados ao checkout de forma segura, substituindo o antigo `checkout.liquid`.

48. O que é o Remix e o Hydrogen?
Hydrogen é o framework headless da Shopify (baseado em React Router/Remix) e Oxygen é a hospedagem global para ele.

49. O que é commerce headless?
Arquitetura em que o frontend é desacoplado do backend de e-commerce e consome APIs, dando flexibilidade e performance, com mais complexidade.

50. Quando escolher headless em vez de tema Liquid?
Quando se precisa de UX altamente customizada, múltiplos canais ou performance extrema; para a maioria das lojas, temas OS 2.0 bastam.

51. O que é o Shopify Polaris?
Design system oficial para criar interfaces de apps consistentes com o Admin.

52. O que são App Bridge e embedded apps?
App Bridge permite que apps rodem dentro do Admin em iframe, com navegação, toasts e ações integradas.

53. O que é o Ajax Cart API?
Endpoints (`/cart/add.js`, `/cart/change.js`, `/cart.js`) para manipular o carrinho via JavaScript sem recarregar a página.

54. O que é a Section Rendering API?
Permite buscar o HTML renderizado de seções por requisição, atualizando partes da página sem recarregar.

55. O que é o Shopify Markets?
Recurso para vender internacionalmente, com moedas, idiomas, domínios e preços por mercado.

56. Como lidar com multilíngue e multimoeda?
Com Markets, arquivos `locales` e o filtro `t` no Liquid, e apps de tradução; moedas locais via Shopify Payments ou conversores.

57. O que é o `Liquid render` versus `include`?
`render` isola o escopo do snippet e é o recomendado; `include` compartilha variáveis e está obsoleto.

58. Como otimizar a performance de um tema?
Imagens responsivas com `image_url` e `srcset`, lazy loading, minimizar apps/JS, fontes otimizadas e CSS crítico.

59. Como medir performance?
Lighthouse, Web Vitals, relatório de velocidade da loja e Theme Inspector for Chrome para perfilar Liquid.

60. O que são `metafield` definitions?
Definições (nome, tipo, validações) de metacampos no admin, permitindo uso no editor de temas e fixando o tipo de dado.

61. Como acessar um metafield no Liquid?
`{{ product.metafields.custom.garantia.value }}`, usando namespace e chave.

62. O que são Collections inteligentes (automated)?
Coleções preenchidas por regras (tag, preço, tipo), atualizadas automaticamente.

63. O que são Selling Plans?
Estrutura para assinaturas e compras recorrentes, pré-venda e planos de pagamento.

64. Como integrar Shopify com ERP e logística?
Via Admin API, webhooks e apps de integração, ou middleware/filas para sincronizar estoque, pedidos e rastreio.

65. O que é o Bulk Operations API?
API GraphQL para executar consultas ou mutações massivas de forma assíncrona, com resultados em JSONL.

66. O que são fulfillment e fulfillment orders?
Fulfillment é o envio dos itens de um pedido; fulfillment orders representam o trabalho a ser cumprido por cada local.

67. Como funciona o controle de estoque?
Por localização (inventory levels), com itens de inventário rastreados por variante e ajustes via Admin ou API.

68. Como testar pagamentos?
Com a opção Bogus Gateway em lojas de desenvolvimento ou modo de teste do gateway, usando cartões de teste.

Nível Avançado (32 perguntas)

69. Como estruturar um app Shopify moderno?
Usar o template Remix/React Router com Shopify CLI, autenticação por token exchange, Prisma para sessões, Polaris/App Bridge e extensões.

70. Como lidar com GraphQL cost e paginação?
Usar cursor-based pagination (`first`, `after`, `pageInfo`), monitorar `extensions.cost` e reduzir complexidade das queries.

71. Como garantir idempotência no processamento de webhooks?
Armazenar o `X-Shopify-Webhook-Id`/evento processado, responder rápido (200) e processar em fila, tolerando reentregas e ordem inconsistente.

72. Por que processar webhooks em fila assíncrona?
A Shopify exige resposta rápida e reenvia em falha; filas (SQS, BullMQ) evitam timeouts e permitem reprocessamento.

73. Como implementar uma Shopify Function de desconto?
Gerar extensão com CLI, definir `input query` GraphQL, escrever a lógica em JS/Rust que retorna operações de desconto e configurar via mutation/admin UI.

74. Quais as limitações das Functions?
Execução em WASM com limite estrito de tempo, instruções e memória, sem acesso a rede, operando só com os dados da query de entrada.

75. Como migrar do `checkout.liquid` para extensibilidade?
Substituir customizações por Checkout UI Extensions, Functions, branding via Checkout Editor e Web Pixels para rastreamento.

76. O que são Web Pixels?
Mecanismo sandboxed para executar scripts de analytics/marketing no storefront e checkout sem acessar o DOM diretamente.

77. Como funciona o Customer Account API?
API GraphQL para experiências de conta do cliente em headless, com OAuth/PKCE para login e acesso a pedidos e dados.

78. Como proteger um app contra vulnerabilidades comuns?
Validar HMAC e session tokens, mínimos escopos, sanitizar entradas, proteger tokens, usar HTTPS e seguir os requisitos de privacidade (GDPR/LGPD).

79. Quais são os webhooks obrigatórios de privacidade?
Apps públicos devem tratar `customers/data_request`, `customers/redact` e `shop/redact` para conformidade com leis de proteção de dados.

80. Como estruturar uma loja multi-região?
Shopify Markets com domínios/subpastas por país, hreflang, catálogos e preços por mercado, e conformidade fiscal local.

81. Como funciona o Shopify Hydrogen com cache?
Estratégias de cache (`CacheShort`, `CacheLong`, SWR) para consultas da Storefront API, combinadas com Oxygen e CDN.

82. Como lidar com busca e filtros avançados?
Storefront Filtering API/Search & Discovery app para filtros nativos, ou provedores como Algolia/Searchspring para relevância avançada.

83. Como projetar integração de sincronização de estoque confiável?
Webhooks + reconciliação periódica via Bulk Operations, idempotência, tratamento de conflitos e monitoramento de falhas.

84. Como automatizar o deploy de temas?
GitHub Actions com Shopify CLI, ambientes de staging/produção, integração GitHub nativa por branch e testes com Theme Check/Lighthouse CI.

85. O que é o Theme Check?
Linter para temas Shopify que verifica boas práticas, performance e erros de Liquid e JSON.

86. Como depurar lentidão causada por Liquid?
Usar Shopify Theme Inspector, reduzir loops aninhados, evitar `forloop` pesado e mover lógica para metafields ou JS assíncrono.

87. Como carregar conteúdo de forma assíncrona sem prejudicar SEO?
Renderizar o conteúdo essencial no HTML do servidor e usar a Section Rendering API ou fetch apenas para conteúdo secundário.

88. Como tratar variantes e dados complexos em headless?
Consultar `selectedOptions`, `availableForSale` e `variantBySelectedOptions` na Storefront API e manter seleção em URL/estado.

89. Como funciona o checkout com Cart API e `checkoutUrl`?
O carrinho é criado via Storefront API (`cartCreate`) e o cliente é redirecionado a `checkoutUrl`, hospedado e seguro pela Shopify.

90. O que são Order Editing e Draft Orders?
Draft Orders criam pedidos manuais ou B2B antes do pagamento; Order Editing altera itens de pedidos existentes via API.

91. Como funciona o B2B na Shopify?

Empresas, locais, catálogos e termos de pagamento específicos, com preços por cliente e pedidos de rascunho (Plus).

92. Como estruturar versionamento da API?

A Shopify lança versões trimestrais e mantém cada uma por cerca de 12 meses; defina a versão no endpoint e planeje atualizações regulares.

93. Como lidar com picos de tráfego (Black Friday)?

Cache e CDN, otimização de imagens e scripts, testes de carga no que for possível, filas para integrações e monitoramento.

94. Como monitorar e observar um app Shopify?

Logs estruturados, métricas, alertas (Sentry/Datadog), rastreio de falhas de webhooks e health checks de integrações.

95. Como lidar com migração de plataforma para Shopify?

Mapear dados (produtos, clientes, pedidos), importar por CSV/Bulk API, configurar redirects 301 para preservar SEO e testar o fluxo de compra.

96. O que são Metaobject entries e como usá-los em headless?

Registros de metaobjects consultáveis pela Storefront API, úteis como CMS leve para conteúdo estruturado.

97. O que são Shopify Scripts e por que migrar?

Scripts em Ruby para Plus, descontinuados em favor das Shopify Functions, mais performáticas e escaláveis.

98. Como equilibrar apps de terceiros e código customizado?

Avaliar impacto em performance e manutenção; usar apps para commodities (avaliações, e-mail) e código próprio para diferenciais de negócio.

99. Como projetar um app de assinaturas?

Usar Selling Plans API e contratos de assinatura, gerenciar cobranças recorrentes (billing attempts), falhas de pagamento e portal do cliente.

100. Como garantir acessibilidade e conformidade em uma loja?

HTML semântico, contraste, navegação por teclado, alt text, testes com axe/Lighthouse e atenção a leis locais (como LBI no Brasil).

6. Next.js

Nível Básico (34 perguntas)

1. O que é Next.js?
Framework React full-stack da Vercel que oferece roteamento, renderização no servidor, otimizações e API routes em uma só ferramenta.

2. Quais as vantagens do Next.js sobre React puro?
Roteamento por arquivos, SSR/SSG/ISR, otimização de imagens e fontes, melhor SEO, code splitting automático e backend integrado.

3. O que é o App Router?
Sistema de roteamento moderno baseado na pasta `app/`, que usa React Server Components, layouts aninhados e streaming.

4. O que é o Pages Router?
Sistema clássico baseado na pasta `pages/`, com `getServerSideProps` e `getStaticProps`; ainda suportado.

5. Como criar uma rota no App Router?
Criando uma pasta em `app/` com um arquivo `page.tsx`; o nome da pasta vira o segmento da URL.

6. O que é um `layout.tsx`?
Componente que envolve páginas filhas e persiste entre navegações, sem re-renderizar. O layout raiz define `<html>` e `<body>`.

7. O que são rotas dinâmicas?
Segmentos entre colchetes, como `app/blog/[slug]/page.tsx`, que capturam parâmetros da URL.

8. O que são catch-all routes?
`[...slug]` captura vários segmentos; `[[...slug]]` os torna opcionais.

9. O que são route groups?
Pastas entre parênteses, como `(marketing)`, que organizam rotas sem afetar a URL.

10. O que é o componente `Link`?
`next/link` realiza navegação no cliente com prefetch automático das rotas visíveis.

11. Como navegar programaticamente?
Com `useRouter` de `next/navigation` (`router.push`) ou `redirect()` em Server Components.

12. O que é o componente `Image`?
`next/image` otimiza imagens automaticamente: redimensionamento, formatos modernos (WebP/AVIF), lazy loading e prevenção de layout shift.

13. O que é `next/font`?
Otimiza fontes (Google ou locais), hospedando-as no próprio domínio e eliminando requisições externas e layout shift.

14. O que são Server Components?
Componentes padrão no App Router, renderizados no servidor, sem enviar JavaScript ao cliente e com acesso direto a dados.

15. O que é `'use client'`?
Diretiva que marca um arquivo e suas importações como Client Components, permitindo estado, efeitos e eventos.

16. Quando usar Client Components?
Quando precisa de interatividade, Hooks (`useState`, `useEffect`), eventos do navegador ou APIs como `localStorage`.

17. Como buscar dados em Server Components?
Usando `async/await` diretamente no componente, com `fetch` ou acesso direto ao banco de dados.

18. O que são Route Handlers?
Endpoints de API definidos em `route.ts` dentro de `app/`, com funções `GET`, `POST`, etc., usando Request/Response padrão.

19. O que são variáveis de ambiente no Next.js?
Definidas em `.env*`; só as prefixadas com `NEXT_PUBLIC_` são expostas ao navegador.

20. O que é o `loading.tsx`?
UI de carregamento exibida automaticamente (via Suspense) enquanto a página ou segmento carrega.

21. O que é o `error.tsx`?
Error boundary de um segmento, exibindo fallback quando ocorre erro; deve ser Client Component.

22. O que é o `not-found.tsx`?
UI exibida quando `notFound()` é chamado ou a rota não existe.

23. O que é metadata no App Router?
API para definir `title`, `description`, Open Graph e outros via `export const metadata` ou `generateMetadata`.

24. Como criar o projeto?
`npx create-next-app@latest`.

25. O que é SSR?
Renderização no servidor a cada requisição, gerando HTML atualizado para SEO e conteúdo dinâmico.

26. O que é SSG?
Geração estática no build; as páginas são servidas prontas via CDN, muito rápidas.

27. O que é ISR?
Regeneração estática incremental: páginas estáticas são atualizadas em segundo plano após um intervalo ou sob demanda.

28. O que é CSR no Next.js?
Renderização no cliente, em que o componente busca dados no navegador após o carregamento.

29. Como estilizar no Next.js?
CSS Modules, Tailwind CSS, CSS global, Sass ou CSS-in-JS compatível com Server Components.

30. O que é a pasta `public`?
Armazena arquivos estáticos servidos a partir da raiz (`/logo.png`).

31. O que é o arquivo `next.config.js`?
Configuração do Next.js: redirects, rewrites, headers, domínios de imagens, variáveis, etc.

32. O que é Turbopack?
Bundler escrito em Rust, sucessor do Webpack no Next.js, que acelera build e desenvolvimento.

33. Como fazer deploy do Next.js?
Na Vercel (mais simples), ou via Node server (`next start`), Docker, ou saída estática (`output: 'export'`).

34. O que é `generateStaticParams`?
Função que define quais parâmetros de rotas dinâmicas serão gerados estaticamente no build.

Nível Intermediário (34 perguntas)

35. Como funciona o cache de `fetch` no Next.js?
Dependendo da versão, o `fetch` pode ser cacheado ou não por padrão; controla-se com `cache`, `next.revalidate` e `next.tags`. Confira a versão utilizada.

36. Como revalidar dados sob demanda?
Com `revalidatePath` e `revalidateTag` em Server Actions ou Route Handlers.

37. O que são Server Actions?
Funções `'use server'` executadas no servidor e chamadas de formulários ou componentes cliente, para mutações sem criar API manual.

38. Como tratar formulários com Server Actions?
Passando a função à prop `action` do `<form>`, validando dados no servidor e usando `useActionState`/`useFormStatus` para feedback.

39. O que é Middleware no Next.js?
Código (`middleware.ts`, ou `proxy` em versões recentes) executado antes da requisição, para autenticação, redirects, i18n e reescritas.

40. O que é o Edge Runtime?
Ambiente leve e rápido executado próximo ao usuário, com APIs limitadas em relação ao Node.js completo.

41. Qual a diferença entre Node runtime e Edge runtime?
Node tem acesso completo a bibliotecas e APIs; Edge tem menor latência e cold start, mas suporte restrito a módulos.

42. O que são layouts aninhados?
Layouts em subpastas que se compõem em hierarquia, preservando estado e evitando re-renderização ao navegar.

43. O que é `template.tsx`?
Semelhante ao layout, mas cria uma nova instância a cada navegação, reiniciando estado e efeitos.

44. O que são Parallel Routes?
Slots nomeados (`@slot`) que renderizam várias páginas simultaneamente no mesmo layout (dashboards, modais).

45. O que são Intercepting Routes?
Rotas que interceptam navegação para exibir conteúdo no contexto atual (ex.: modal de foto), mantendo URL compartilhável.

46. Como fazer streaming com Suspense?
Envolver partes lentas com `<Suspense fallback>`; o HTML é enviado em fluxo e a UI aparece progressivamente.

47. Como proteger rotas e implementar autenticação?
Com Auth.js (NextAuth), Clerk ou sessões próprias com cookies httpOnly, verificando no middleware e no servidor, não só no cliente.

48. Como funciona o `cookies()` e `headers()`?
APIs para ler cookies e cabeçalhos no servidor; usá-las torna a rota dinâmica.

49. O que é renderização dinâmica vs estática?
Estática gera HTML no build/revalidação; dinâmica renderiza por requisição, ativada ao usar APIs dinâmicas como `cookies`, `headers` ou `searchParams`.

50. O que é Partial Prerendering (PPR)?
Combina um shell estático servido rapidamente com partes dinâmicas transmitidas via Suspense na mesma rota.

51. Como otimizar imagens externas?
Configurar `images.remotePatterns` no `next.config`, e usar `sizes`, `priority` e `placeholder="blur"`.

52. Como configurar redirects e rewrites?
No `next.config.js` com `redirects()` e `rewrites()`, ou em middleware para lógica dinâmica.

53. O que são `generateMetadata` e Open Graph?
`generateMetadata` gera metadados dinâmicos por rota, incluindo tags Open Graph e Twitter para compartilhamento.

54. Como gerar `sitemap.xml` e `robots.txt`?
Criando `sitemap.ts` e `robots.ts` em `app/`, que exportam funções retornando os dados.

55. Como fazer internacionalização (i18n)?
No App Router, com segmentos `[lang]`, middleware para detectar idioma e bibliotecas como next-intl.

56. Como usar variáveis de ambiente em runtime vs build?
`NEXT_PUBLIC_*` são embutidas no build; as demais ficam no servidor e podem ser lidas em runtime.

57. Como compartilhar dados entre Server Components?
Chamar funções de dados repetidamente com `fetch` deduplicado ou `React.cache`, evitando prop drilling e Context no servidor.

58. Posso usar Context em Server Components?
Não; Providers precisam ser Client Components. Envolva a árvore em um componente cliente de Provider.

59. Como passar Server Components como children de Client Components?
Com a prop `children`, que mantém o conteúdo renderizado no servidor dentro do wrapper cliente.

60. O que é `use cache`?
Diretiva de cache de funções/componentes introduzida em versões recentes (Cache Components), que substitui parte dos comportamentos implícitos anteriores.

61. O que são `next/script` e como carregar scripts de terceiros?
Componente que controla estratégia de carregamento (`beforeInteractive`, `afterInteractive`, `lazyOnload`) de scripts externos.

62. Como fazer upload de arquivos?
Via Route Handler/Server Action com `FormData`, ou URLs pré-assinadas para S3/Cloud Storage para evitar passar pelo servidor.

63. Como conectar ao banco de dados?
Usando ORMs como Prisma ou Drizzle em Server Components/Actions/Route Handlers, nunca no código cliente.

64. Como testar aplicações Next.js?
Vitest/Jest + Testing Library para componentes, Playwright/Cypress para E2E, e mocks de rede com MSW.

65. Como lidar com erros globais?
Com `global-error.tsx`, `error.tsx` por segmento e monitoramento (Sentry).

66. O que é `useRouter` vs `usePathname`?
`useRouter` navega e atualiza; `usePathname` lê o caminho atual para estilizar links ativos.

67. O que é `unstable_cache`/cache de dados em Server?
Utilitário (e substitutos mais recentes) para cachear resultados de funções que não usam `fetch`, com tags e revalidação.

68. O que é Static Export?
Gera um site 100% estático (`output: 'export'`), sem recursos que dependem de servidor como Server Actions e middleware.

Nível Avançado (32 perguntas)

69. Como o Next.js divide o bundle?
Automaticamente por rota e por componentes dinâmicos; Server Components não entram no bundle do cliente.

70. O que é `next/dynamic`?
Carrega componentes sob demanda (lazy), com opção de desabilitar SSR (`ssr: false`) para componentes dependentes de `window`.

71. Como funciona o Router Cache no cliente?
Cache em memória de segmentos já visitados/prefetched, que acelera navegações; pode ser invalidado por `router.refresh()` ou revalidações.

72. Explique as camadas de cache do Next.js.
Request Memoization (deduplicação por render), Data Cache (servidor), Full Route Cache (HTML/RSC no build) e Router Cache (cliente); os padrões variam por versão.

73. Como o RSC Payload funciona?
Formato serializado da árvore de Server Components enviado ao cliente para reconciliar a UI e hidratar Client Components.

74. O que são erros de hidratação e como evitá-los?
Divergência entre HTML do servidor e cliente (datas, `window`, IDs); resolvem-se com `useEffect`, `useId` ou `suppressHydrationWarning`.

75. Como evitar waterfalls de dados no App Router?
Disparar `fetch`es em paralelo com `Promise.all`, usar Suspense granulares e iniciar requisições o mais cedo possível (preload).

76. Como implementar autenticação segura end-to-end?
Sessões em cookies httpOnly/secure/SameSite, verificação em Server Actions e Route Handlers (não apenas middleware), CSRF e autorização por recurso.

77. Quais cuidados de segurança com Server Actions?
São endpoints públicos: valide entradas (Zod), autentique e autorize cada ação, evite vazar dados e mantenha o framework atualizado.

78. Como estruturar um monorepo com Next.js?
Turborepo/Nx com apps e pacotes compartilhados (UI, config, tipos), cache de build remoto e `transpilePackages`.

79. Como otimizar Core Web Vitals?
Priorizar LCP com `next/image` `priority`, evitar layout shift, reduzir JS cliente, streaming, fontes otimizadas e CDN.

80. Como implementar ISR em larga escala?
Usar `revalidate` por tempo, revalidação por tags via webhooks do CMS e `generateStaticParams` parcial com `dynamicParams`.

81. Como funciona `dynamicParams`?
Define se rotas não retornadas por `generateStaticParams` são geradas sob demanda (`true`) ou retornam 404 (`false`).

82. Como implementar multi-tenancy?
Middleware reescrevendo subdomínios/domínios para segmentos dinâmicos, isolando dados por tenant no banco e no cache.

83. Como lidar com feature flags e A/B testing?
Decisão no middleware ou servidor (Edge Config, LaunchDarkly), mantendo variações estáticas via rewrites para preservar cache.

84. Como funciona o Middleware e quais suas limitações?
Roda antes do cache/rotas, ideal para decisões leves; evite lógica pesada e acesso a bancos tradicionais, pois o runtime é restrito.

85. Como fazer streaming de respostas de IA?
Route Handler retornando `ReadableStream`/SSE, consumido no cliente (ex.: AI SDK com `useChat`).

86. Como implementar WebSockets/real-time?
Vercel serverless não mantém conexões longas; use serviços externos (Pusher, Ably, Supabase) ou servidor customizado/SSE.

87. Como self-host de Next.js com escala horizontal?
Container Node/Docker com `output: 'standalone'`, cache compartilhado (handler customizado com Redis), CDN e balanceador.

88. O que é `instrumentation.ts`?
Arquivo que executa código na inicialização do servidor, usado para OpenTelemetry e monitoramento.

89. Como observar e monitorar em produção?
OpenTelemetry, Sentry, logs estruturados, Web Vitals via `useReportWebVitals` e métricas do host.

90. Quando preferir Pages Router em vez de App Router?
Projetos legados, bibliotecas incompatíveis com RSC ou equipes que precisam de menor mudança; novos projetos tendem ao App Router.

91. Como migrar do Pages Router para App Router?
Incrementalmente: ambos coexistem; mover rotas uma a uma, trocar `getServerSideProps` por Server Components e adaptar layouts e metadados.

92. O que são Route Segment Config options?
Exports como `dynamic`, `revalidate`, `runtime`, `maxDuration` e `fetchCache` que controlam o comportamento de renderização de um segmento.

93. Como tratar CSS-in-JS no App Router?
Bibliotecas devem suportar RSC (registro de estilos no servidor) ou usar alternativas como CSS Modules, Tailwind ou vanilla-extract.

94. Como otimizar builds longos?

Turbopack, cache de build, ISR em vez de gerar tudo estático, `generateStaticParams` parcial e monorepo com cache remoto.

95. O que são Cache Components e a abordagem "dinâmico por padrão"?

Modelo mais recente em que o código é dinâmico por padrão e o cache é explícito com `use cache`, combinado a PPR; veja a documentação da sua versão.

96. Como diferenciar Server e Client de forma segura para segredos?

Usar `server-only` para impedir importação acidental no cliente e nunca expor chaves sem `NEXT_PUBLIC_`.

97. Como lidar com SEO avançado?

Metadados dinâmicos, JSON-LD, canonical, sitemaps, SSR/SSG de conteúdo, `hreflang` e URLs consistentes.

98. Como controlar headers de segurança?

Via `headers()` no `next.config.js` ou middleware: CSP, HSTS, X-Frame-Options e Referrer-Policy.

99. Como lidar com grandes listas e paginação?

Paginação por cursor no servidor, `searchParams` como fonte de verdade, infinite scroll com Client Component e virtualização.

100. Como decidir entre Next.js e outras alternativas (Remix, Astro)?

Next.js para apps full-stack React com ecossistema vasto; Remix/React Router para foco em padrões web; Astro para sites de conteúdo com pouco JS.

7. Vaga Fullstack React + Node.js

Nível Básico (34 perguntas)

1. O que é Node.js?
Ambiente de execução JavaScript baseado no motor V8 que roda fora do navegador, orientado a eventos e com I/O não bloqueante.

2. Node.js é single-thread?
O código JavaScript roda em uma thread principal, mas I/O é delegado ao libuv (thread pool e SO), permitindo concorrência. Para CPU intensiva usam-se worker threads.

3. O que é o npm?
Gerenciador de pacotes do Node, com registro público e o arquivo `package.json` para dependências e scripts.

4. Qual a diferença entre `dependencies` e `devDependencies`?
`dependencies` são necessárias em produção; `devDependencies` apenas em desenvolvimento (testes, lint, build).

5. O que é o `package-lock.json`?
Arquivo que fixa as versões exatas instaladas para garantir builds reproduzíveis.

6. O que é Express?
Framework web minimalista para Node, que facilita criar rotas, middlewares e APIs HTTP.

7. O que é um middleware?
Função com acesso a `req`, `res` e `next` que processa requisições em cadeia (logs, autenticação, parsing, erros).

8. O que é uma API REST?
Estilo arquitetural que expõe recursos via URLs e métodos HTTP (GET, POST, PUT, PATCH, DELETE), de forma stateless.

9. Quais os principais métodos HTTP e seus usos?
GET lê; POST cria; PUT substitui; PATCH atualiza parcialmente; DELETE remove.

10. O que significam os códigos 200, 201, 400, 401, 403, 404 e 500?
Sucesso, criado, requisição inválida, não autenticado, proibido, não encontrado e erro interno do servidor.

11. Qual a diferença entre 401 e 403?
401: não autenticado (identidade ausente/inválida); 403: autenticado, mas sem permissão.

12. O que é JSON e por que é usado em APIs?
Formato leve e legível de troca de dados, nativo em JavaScript, usado entre frontend e backend.

13. Como um React app se comunica com um backend Node?
Por requisições HTTP (fetch/axios/React Query) a endpoints REST ou GraphQL, trocando JSON.

14. O que é CORS e como configurar no Express?
Controle de requisições entre origens diferentes; configura-se com o middleware `cors`, definindo origens, métodos e credenciais permitidas.

15. O que são variáveis de ambiente?
Valores de configuração externos ao código (portas, URLs, segredos), lidos por `process.env` e geralmente definidos em `.env`.

16. **Por que nunca commitar o `.env`?**
    Contém segredos (chaves, senhas); deve ir ao `.gitignore`, com um `.env.example` sem valores reais.

17. **O que é `async/await` no Node?**
    Sintaxe para lidar com Promises de forma sequencial, essencial para I/O como banco e HTTP.

18. **Como ler o corpo de uma requisição JSON no Express?**
    Com `app.use(express.json())` e acesso via `req.body`.

19. **Como definir parâmetros de rota e de query?**
    Rota: `/users/:id` com `req.params.id`; query: `/users?page=2` com `req.query.page`.

20. **O que é um ORM?**
    Mapeia tabelas do banco para objetos, simplificando consultas: Prisma, Sequelize, TypeORM, Drizzle.

21. **Qual a diferença entre SQL e NoSQL?**
    SQL é relacional, com esquema e joins (PostgreSQL, MySQL); NoSQL tem modelos flexíveis (MongoDB, Redis), ideais para certos casos de uso.

22. **O que é MongoDB?**
    Banco NoSQL orientado a documentos JSON (BSON), flexível e escalável horizontalmente.

23. **O que é autenticação e autorização?**
    Autenticação confirma quem é o usuário; autorização define o que ele pode fazer.

24. **O que é JWT?**
    JSON Web Token: token assinado contendo claims, usado para autenticação stateless; deve ter expiração e ser transmitido com segurança.

25. **Por que fazer hash de senhas?**
    Para nunca armazenar a senha em texto puro; usa-se bcrypt/argon2 com salt, que são lentos de propósito.

26. **O que é um proxy no Vite/CRA para desenvolvimento?**
    Configuração que redireciona chamadas `/api` do frontend para o backend local, evitando CORS em desenvolvimento.

27. **O que é o `useEffect` típico para buscar dados da API?**
    Efeito que chama `fetch` ao montar o componente e atualiza o estado com a resposta, tratando loading e erro.

28. **O que é um monorepo?**
    Um único repositório que contém vários projetos (frontend, backend, pacotes compartilhados).

29. **O que é nodemon?**
    Ferramenta de desenvolvimento que reinicia o servidor Node ao detectar alterações nos arquivos.

30. **Como estruturar um projeto Express?**
    Camadas como `routes`, `controllers`, `services`, `models/repositories` e `middlewares`, separando responsabilidades.

31. **O que é o padrão MVC?**
    Separa Model (dados), View (interface) e Controller (lógica de entrada); em APIs, a "view" costuma ser o JSON.

32. **O que é HTTPS e por que importa?**
    HTTP sobre TLS, criptografa o tráfego e garante integridade e autenticidade do servidor.

33. Como tratar erros em Express?
Com middleware de erro `(err, req, res, next)` ao final da cadeia, e `try/catch` (ou wrappers) em handlers assíncronos.

34. O que é validação de entrada e por que fazer no backend?
Garantir que os dados recebidos são válidos e seguros; o frontend pode ser burlado, então a validação do servidor é obrigatória (Zod, Joi).

Nível Intermediário (33 perguntas)

35. Como funciona o Event Loop do Node.js?
Processa fases (timers, pending, poll, check, close), com microtasks (`process.nextTick`, Promises) entre elas, garantindo I/O não bloqueante.

36. Como evitar bloquear o event loop?
Evitar operações síncronas pesadas, mover CPU intensiva para worker threads ou serviços/filas e usar streams para grandes dados.

37. O que são streams no Node?
Interfaces para processar dados em partes (leitura, escrita, transformação), economizando memória em arquivos e respostas grandes.

38. O que é cluster/worker threads?
`cluster` cria processos para usar vários núcleos; `worker_threads` executa JS em threads separadas para CPU intensiva.

39. Como implementar autenticação com JWT em React + Node?
Login valida credenciais e emite token (access curto + refresh); o cliente o envia (preferencialmente cookie httpOnly); middleware verifica a assinatura e injeta o usuário.

40. Onde armazenar o token no frontend?
Preferencialmente em cookies httpOnly + Secure + SameSite, que mitigam XSS; localStorage é vulnerável a roubo por XSS.

41. O que é refresh token e rotação?
Token de longa duração usado para obter novos access tokens; a rotação o substitui a cada uso, detectando reutilização indevida.

42. Como implementar rotas protegidas no React?
Um componente `ProtectedRoute` verifica a sessão e redireciona ao login; o backend sempre revalida, pois o frontend não é confiável.

43. Como implementar autorização por papéis (RBAC)?
Associar papéis/permissões ao usuário e verificar em middleware por rota ou recurso no servidor.

44. O que é Prisma e quais suas vantagens?
ORM TypeScript-first com schema declarativo, migrations, cliente tipado e boa experiência de desenvolvimento.

45. O que são migrations?
Scripts versionados que evoluem o esquema do banco de forma controlada e reprodutível.

46. O que é o problema N+1?
Executar uma consulta por item de uma lista em vez de uma consulta agregada; resolve-se com `include/join`, batching ou DataLoader.

47. O que são índices no banco e quando usá-los?
Estruturas que aceleram buscas em colunas consultadas com frequência, com custo em escrita e espaço.

48. O que são transações?
Conjunto de operações atômicas (ACID): ou todas são aplicadas ou nenhuma.

49. O que é paginação e quais tipos existem?
Divide resultados em partes: offset/limit (simples, ruim em grandes volumes) e cursor-based (mais eficiente e estável).

50. O que é o TanStack Query e por que usar com uma API Node?
Gerencia cache, loading, erros, revalidação e mutações do estado do servidor no React, eliminando `useEffect` manual.

51. Como compartilhar tipos entre frontend e backend?
Monorepo com pacote de tipos/schemas (Zod) compartilhado, ou geração a partir de OpenAPI/tRPC.

52. O que é tRPC?
Biblioteca que permite chamar funções do servidor com tipagem ponta a ponta em TypeScript, sem schema ou geração de código.

53. O que é GraphQL e quando usar?
Linguagem de consulta que permite ao cliente pedir exatamente os dados necessários; útil com muitos clientes e dados relacionados, com mais complexidade que REST.

54. Como versionar uma API REST?
Por prefixo (`/v1/`), cabeçalhos ou media types, evitando quebrar clientes existentes.

55. O que é rate limiting e como aplicar?
Limitar o número de requisições por cliente/IP em um intervalo (`express-rate-limit`, Redis), mitigando abuso e DoS.

56. O que é Helmet?
Middleware que define cabeçalhos HTTP de segurança no Express.

57. Como prevenir SQL Injection?
Usando queries parametrizadas/ORM e nunca concatenando entrada do usuário em SQL.

58. Como prevenir XSS?
React escapa por padrão; evite `dangerouslySetInnerHTML`, sanitize HTML (DOMPurify), use CSP e cookies httpOnly.

59. O que é CSRF e como se proteger?
Requisição forjada usando a sessão do usuário; use cookies `SameSite`, tokens CSRF e verificação de origem.

60. Como fazer upload de arquivos com React + Node?
`FormData` no frontend e `multer` (ou URL pré-assinada para S3) no backend, validando tipo e tamanho.

61. O que é WebSocket e quando usar?
Comunicação bidirecional persistente (Socket.IO/ws), útil para chats, notificações e dados em tempo real.

62. Como testar a API Node?
Com Jest/Vitest e Supertest para endpoints, bancos em memória ou containers de teste e mocks de serviços externos.

63. Como testar o frontend e a integração?
React Testing Library para componentes, MSW para simular a API e Playwright/Cypress para fluxos E2E.

64. O que é Docker e por que usar?
Containers empacotam app e dependências, garantindo consistência entre ambientes; usa-se `Dockerfile` e `docker-compose` para app + banco.

65. O que é CI/CD para esse stack?
Pipeline (GitHub Actions) que roda lint, testes e build a cada push e faz deploy automático nas branches certas.

66. Como logar corretamente em Node?
Logs estruturados em JSON com níveis (pino/winston), correlação por request ID e sem dados sensíveis.

67. O que é graceful shutdown?
Encerrar o servidor ao receber SIGTERM concluindo requisições em andamento e fechando conexões (banco, filas).

Nível Avançado (33 perguntas)

68. Como projetaria uma arquitetura escalável para React + Node?
Frontend em CDN, backend stateless atrás de load balancer, cache (Redis), banco com réplicas de leitura, filas para tarefas assíncronas e observabilidade.

69. Como escalar o Node horizontalmente?
Múltiplas instâncias stateless atrás de balanceador, sessões externas (Redis/JWT) e autoscaling baseado em CPU/latência.

70. Como projetar uma API idempotente?
Usar chaves de idempotência (`Idempotency-Key`) em POSTs críticos como pagamentos, armazenando o resultado da primeira execução.

71. Quando usar filas (BullMQ, RabbitMQ, SQS)?
Para tarefas demoradas ou falhas toleráveis (e-mails, relatórios, integrações), desacoplando e permitindo retries e escala de workers.

72. Como implementar cache em múltiplas camadas?
Cache de CDN, HTTP (`Cache-Control`, ETag), cache de aplicação (Redis) e cache do cliente (React Query), com invalidação definida.

73. Como lidar com invalidação de cache?
TTL, invalidação por evento/tag e estratégias como write-through ou cache-aside; evitar inconsistências com chaves bem planejadas.

74. Como projetar um sistema de autenticação robusto?
OAuth2/OIDC, MFA, refresh rotation, revogação de sessões, rate limit em login, auditoria e armazenamento seguro de segredos.

75. Como implementar OAuth/Login social?
Fluxo Authorization Code com PKCE; o backend troca o código por tokens, cria a sessão e nunca expõe o client secret ao frontend.

76. Como lidar com concorrência e race conditions no banco?
Transações com níveis de isolamento adequados, locks otimistas (versão) ou pessimistas e constraints únicas.

77. O que é o padrão Repository e Service Layer?
Repository isola o acesso a dados; Service concentra regras de negócio; facilita testes e troca de tecnologias.

78. O que é Clean/Hexagonal Architecture?
Organização em camadas com dependências apontando para o domínio, isolando regras de negócio de frameworks e infraestrutura.

79. O que é microservices vs monolito modular?
Microservices separam serviços implantáveis independentemente, com custo operacional; monolito modular mantém um deploy com módulos bem delimitados, geralmente melhor ponto de partida.

80. Como garantir consistência entre serviços?
Eventos e padrões como Saga/Outbox, idempotência e consistência eventual em vez de transações distribuídas.

81. Como observar e depurar produção?
Logs estruturados, métricas (Prometheus/Grafana), tracing distribuído (OpenTelemetry), alertas e monitoramento de erros (Sentry).

82. Como diagnosticar vazamento de memória em Node?
Usar `--inspect`, heap snapshots no Chrome DevTools, `clinic.js` e checar listeners, caches ilimitados e closures retendo referências.

83. Como otimizar performance de uma API Node?
Reduzir consultas e payloads, índices, cache, compressão, paginação, connection pooling e profiling (flamegraphs).

84. O que é connection pooling?
Reutilizar um conjunto limitado de conexões ao banco, evitando o custo de abri-las e sobrecarregá-lo.

85. Como lidar com validação e contratos de API?
Schemas compartilhados (Zod/OpenAPI), geração de clientes tipados, testes de contrato e versionamento cuidadoso.

86. Como implementar paginação infinita no React com API Node?
Cursor no backend e `useInfiniteQuery` no frontend, com IntersectionObserver para carregar mais itens.

87. Como implementar atualizações otimistas?
Atualizar a UI imediatamente (React Query `onMutate` ou `useOptimistic`), reverter em caso de erro e revalidar ao concluir.

88. Como evitar re-renders e lentidão em apps React grandes?
Dividir estado, memoização seletiva, virtualização de listas, code splitting e medir com Profiler.

89. Como funciona deploy sem downtime?
Estratégias blue/green ou rolling, health checks, migrações compatíveis em duas fases (expand/contract) e feature flags.

90. Como fazer migrações de banco seguras?
Alterações retrocompatíveis em etapas, executar em baixa carga, backups e rollback planejado.

91. Como proteger segredos e dependências?
Gerenciadores de segredos, rotação de chaves, `npm audit`/Dependabot, lockfiles e revisão de pacotes (supply chain).

92. Como lidar com LGPD/GDPR na aplicação?

Minimizar dados, consentimento, direito de acesso e exclusão, criptografia, logs sem dados pessoais e registros de tratamento.

93. Como projetar uma API para processar pagamentos?

Integrar gateway (Stripe/Mercado Pago), confirmar por webhooks assinados, usar idempotência, nunca guardar dados de cartão e tratar estados assíncronos.

94. Como fazer real-time em escala?

Socket.IO com adaptador Redis, serviços gerenciados (Pusher/Ably) ou SSE; considerar reconexão, backpressure e autenticação.

95. Como organizar um monorepo fullstack?

pnpm/Turborepo com `apps/web`, `apps/api` e `packages/{ui,types,config}`, pipeline de build com cache e versionamento compartilhado.

96. Como você abordaria a decisão entre REST, GraphQL e tRPC?

REST para APIs públicas e simples, GraphQL para clientes variados com dados relacionados, tRPC para times full TypeScript com controle de ambos os lados.

97. Como conduzir code review e liderar tecnicamente?

Foco em clareza, riscos e arquitetura; feedback objetivo e respeitoso, padrões automatizados (lint, testes) e compartilhamento de conhecimento.

98. Fale sobre um desafio técnico difícil que você resolveu.

Use o método STAR: Situação, Tarefa, Ação e Resultado, com métricas (ex.: reduziu latência em 40%) e aprendizados. Prepare uma história real antes da entrevista.

99. Como você priorizaria entregas sob prazo apertado?

Alinhar o escopo mínimo de valor (MVP), identificar riscos, comunicar trade-offs ao time/produto e registrar dívida técnica assumida.

100. Que perguntas fazer ao entrevistador?

Sobre stack e arquitetura, processo de deploy, cultura de testes e code review, onboarding, métricas de sucesso da função e oportunidades de crescimento.